«Программа для геодезического контроля и мониторинга объектов инфраструктуры на базе цифрового двойника и оптико-электронных систем измерений». Листинги исходного кода

Модуль сбора и первичной обработки данных датчиков

Файл BBBsoft/src/**main.py**

```
# -*- coding: utf-8 -*-
#!/usr/bin/env python
import os, sys, argparse, logging, time, asyncio
from logging.handlers import RotatingFileHandler
from datetime import datetime, timezone
import zmq.asyncio
import math, random, statistics
import yaml, importlib.metadata
from collections import deque
from typing import Dict, Any, Optional, Deque, Tuple,
  TYPE_CHECKING

from src.sensors.inclinometer import Inclinometer
from src.communication.zmq_handler import ZMQHandler

try:
    from src import _build_info
except ImportError:
    class _BuildInfoMock:
        VERSION = "0.0.0-unknown-mock" # Отличаем от
  PackageNotFound
        COMMIT_HASH = "unknown"
        COMMIT_DATE = "unknown"
    _build_info = _BuildInfoMock()
```

```
    print("ПРЕДУПРЕЖДЕНИЕ: Не удалось загрузить информацию о сборке (src/_build_info.py). Отображается базовая версия.", file=sys. stderr)

async def measure_inclinometer_frequency(config: Dict[str, Any], duration_sec: int = 2) -> Optional[float]:
    """Измеряет фактическую частоту поступления данных с инклинометра."""
    logger = logging.getLogger("MeasureInclinometer")
    inclinometer = None
    try:
        inc_config = config['sensors']['inclinometer']
        device = inc_config.get('device')
        baudrate = inc_config.get('baudrate', 115200)
        message_header_hex = inc_config.get('message_header_hex')
        if not device or not message_header_hex:
            logger.error("Недостаточно параметров конфигурации для измерения частоты инклинометра.")
            return None

        logger.info(f"Начинаем измерение частоты инклинометра на {device} в течение {duration_sec} сек.")
        inclinometer = Inclinometer(device=device, baudrate=baudrate, message_header_hex=message_header_hex)
        connected = await inclinometer.connect()
        if not connected:
            logger.error("Не удалось подключиться к инклинометру для измерения частоты.")
            return None
```

```
        logger.debug("Measure Freq: Connection successful, entering read loop.")

        count = 0
        start_time = time.monotonic()
        end_time = start_time + duration_sec
        last_log_time = start_time

        while time.monotonic() < end_time:
            current_loop_time = time.monotonic()
            if current_loop_time - last_log_time > 1.0:
                logger.debug(f"Measure Freq Loop: Time left {end_time - current_loop_time:.1f}s, Count={count}")
                last_log_time = current_loop_time

            try:
                logger.debug(f"Measure Freq Loop: Calling read_data_async...")
                read_start = time.monotonic()
                data = await inclinometer.read_data_async()
                read_duration = time.monotonic() - read_start
                logger.debug(f"Measure Freq Loop: read_data_async finished in {read_duration:.4f}s. Data received: {bool(data)}")

                if data:
                    count += 1
                else:
                    # Небольшая пауза, если данных нет: 1 мс
                    await asyncio.sleep(0.001)
            except Exception as e:
```

```
                logger.error(f"Ошибка при чтении инклинометра во время измерения частоты: {e}")
                # При серьезной ошибке прерываем измерение
                if not inclinometer.is_connected():
                    logger.error("Потеряно соединение во время измерения.")
                    break
                await asyncio.sleep(0.1)

        actual_duration = time.monotonic() - start_time
        # Избегаем деления на ноль, если измерение прервалось мгновенно
        frequency = count / actual_duration if actual_duration > 0.001 else 0
        logger.info(f"Измерение завершено. Получено {count} сообщений за {actual_duration:.2f} сек. Расчетная частота: {frequency:.2f} Гц")
        return frequency

    except Exception as e:
        logger.exception("Ошибка во время измерения частоты инклинометра")
        return None
    finally:
        if inclinometer:
            try:
                await inclinometer.close()
                logger.info("Ресурс Inclinometer для измерения частоты освобожден.")
            except Exception as e:
                logger.error(f"Ошибка при закрытии Inclinometer: {e}")

class MeasurementSystem:
```

```
    adc: Optional['ADC']
    inclinometer: Optional[Inclinometer]    # Inclinometer импортирован
    zmq_handler: Optional[ZMQHandler]       # ZMQHandler импортирован

    def __init__(self, config: Dict[str, Any], simulate: bool = False):
        """Инициализация асинхронной системы измерений."""
        self.config = config
        self.simulate = simulate
        self._setup_logging()
        self.logger.info(f"Асинхронная система измерений инициализируется... Режим( симуляции: ВКЛ{'' if simulate else ВЫКЛ''})")

        # Инициализация RCpy важно( сделать до ADC), только если не симуляция
        if not self.simulate:
            logger.debug("Before first _init_rcpy call in __init__")
            self._init_rcpy()
            logger.debug("After first _init_rcpy call in __init__")
        else:
            self.logger.info("Режим симуляции: Пропуск инициализации RCpy.")

        # --- Инициализация компонентов синхронная( часть) ---
        if self.simulate:
            self.logger.info("Режим симуляции: Пропуск инициализации реальных сенсоров.")
            self.adc = None
```

```
            self.inclinometer = None
        else:
            # Инициализируем реальные сенсоры только если не симуляция
            self.adc = self._init_adc()
            self.inclinometer = self._init_inclinometer()

        # Инициализируем ZMQ всегда
        self.zmq_handler = self._init_zmq()

        # Параметры обработки
        # Получаем значения из конфига и преобразуем в float
        try:
            processing_freq_str = str(self.config.get('processing', {}).get('frequency_hz', '1.0')).replace(',', '.')
            self.processing_freq_hz = float(processing_freq_str)
        except ValueError:
            self.logger.warning(f"Не удалось преобразовать processing frequency '{processing_freq_str}' в float. Используется 1.0.")
            self.processing_freq_hz = 1.0

        try:
            aggregation_period_str = str(self.config.get('processing', {}).get('aggregation_period_sec', '1.0')).replace(',', '.')
            self.aggregation_period_sec = float(aggregation_period_str)
        except ValueError:
            self.logger.warning(f"Не удалось преобразовать aggregation period '{
```

```
aggregation_period_str}' в float. Используется 1.0.")
            self.aggregation_period_sec = 1.0

        if self.processing_freq_hz <= 0: self.processing_freq_hz = 1.0
        if self.aggregation_period_sec <= 0: self.aggregation_period_sec = 1.0
        self.processing_interval_sec = 1.0 / self.processing_freq_hz

        # Определение размеров очередей
        # Получаем частоту ADC и преобразуем в float
        try:
            adc_freq_str = str(self.config.get('sensors', {}).get('adc', {}).get('frequency_hz', '10.0')).replace(',', '.')
            self.adc_freq_hz = float(adc_freq_str)
        except ValueError:
            self.logger.warning(f"Не удалось преобразовать ADC frequency '{adc_freq_str}' в float. Используется 10.0.")
            self.adc_freq_hz = 10.0

        if self.adc_freq_hz <= 0: self.adc_freq_hz = 10.0
        self.adc_interval_sec = 1.0 / self.adc_freq_hz
        self.adc_queue_size = max(1, int(self.adc_freq_hz * self.aggregation_period_sec))

        # Частоту инклинометра нужно измерить ДО создания очереди
        self.inclinometer_freq_hz = 5
        self.inclinometer_queue_size = max(1, int(self.inclinometer_freq_hz * self.aggregation_period_sec))
```

```
        # Очереди для данных (timestamp, value)
        # Для ADC храним raw значения
        self.adc_data_queue:
            Deque[Tuple[float, int]] = deque(maxlen=self.adc_queue_size)
        # Для Inclinometer храним полные словари
        self.inclinometer_data_queue:
            Deque[Tuple[float, Dict[str, Any]]] = deque(maxlen=self.inclinometer_queue_size)

        self._tasks = [] # Список для хранения запущенных задач asyncio
        self._shutdown_event = asyncio.Event()

        self.logger.info(f"ADC: частота={self.adc_freq_hzГц}, очередь={self.adc_queue_size} ({self.aggregation_period_secc})")
        self.logger.info("Инициализация завершена синхронная( часть)")

    def _init_rcpy(self):
        """Инициализация RCpy. Вызывается только если НЕ симуляция."""
        self.logger.debug("Вход в _init_rcpy")
        global rcpy # Используем глобальные переменные rcpy, rcpy_adc
        global rcpy_adc
        global rcpy_available # и флаг доступности

        # Импортируем rcpy
        try:
            self.logger.debug("Попытка импорта rcpy...")
            import rcpy
```

```
            import rcpy.adc as rcpy_adc
            rcpy_available = True
            self.logger.debug("rcpy и rcpy.adc импортированы успешно.")
        except ImportError as e:
            self.logger.debug(f"Ошибка импорта rcpy или rcpy.adc: {e}")
            self.logger.error("Модуль rcpy или rcpy.adc не найден! Установите его для работы с оборудованием.")
            rcpy_available = False
            # Не выбрасываем исключение здесь,
            # чтобы позволить работать в режиме симуляции.
            # Но ADC работать не будет.
            self.logger.debug("Выход из _init_rcpy (rcpy недоступен)")
            return # Выходим, если импорт не удался

        # --- Дальнейшая инициализация только если импорт успешен ---
        try:
            self.logger.debug("Получение текущего состояния rcpy...")
            current_state = rcpy.get_state()
            self.logger.debug(f"Текущее состояние rcpy: {current_state}")

            if current_state == rcpy.EXITING:
                self.logger.debug("Состояние EXITING, вызов rcpy.initialize()...")
                rcpy.initialize()
                self.logger.info("RCpy инициализирован.")
            elif current_state == rcpy.RUNNING:
```

```
                self.logger.info("RCpy уже инициализирован.")
            else:
                self.logger.warning(f"RCpy в состоянии {current_state}. Попытка запуска...")
                rcpy.run()
                self.logger.debug("rcpy.run() завершен.")
                # Повторно проверяем состояние после вызова run()
                new_state = rcpy.get_state()
                self.logger.debug(f"Новое состояние rcpy после run(): {new_state}")
                if new_state != rcpy.RUNNING:
                    self.logger.debug(f"Ошибка: не удалось перевести в RUNNING.")
                    raise RuntimeError(f"Не удалось перевести RCpy в состояние RUNNING текущее(: {new_state})")
                self.logger.info(f"RCpy переведен в состояние RUNNING.")
        except Exception as e:
            self.logger.debug(f"Исключение в _init_rcpy: {e}")
            self.logger.exception("Критическая ошибка инициализации RCpy")
            raise RuntimeError(f"Не удалось инициализировать RCpy: {e}")
        finally:
            logger.debug("Выход из _init_rcpy")

    def _init_adc(self) -> Optional['ADC']:
        """Инициализация датчика ADC. Вызывается только если НЕ симуляция.
```

```
            Больше не создает объект ADC, так как используется прямой вызов
            функций. Возвращает None."""
        if not self.simulate and rcpy_available:
            try:
                channel = self.config['sensors']['adc']['channel']
                self.logger.info(f"Конфигурация ADC для канала {channel} найдена.")
                _ = rcpy.adc.get_raw(channel) # Пробное чтение
                self.logger.info(f"Пробное чтение rcpy.adc.get_raw({channel}) успешно.")
            except KeyError as e:
                self.logger.error(f"Отсутствует ключ в конфигурации ADC: {e}. ADC не будет использоваться.")
            except AttributeError:
                self.logger.error("Ошибка доступа к rcpy.adc функциям. Проверьте установку rcpy.")
            except Exception as e:
                self.logger.exception(f"Неожиданная ошибка при проверке ADC канала {channel}: {e}")
        elif not self.simulate:
            self.logger.warning("rcpy недоступен, ADC инициализация пропускается.")
        return None # Возвращаем None, так как объекта ADC больше нет

    def _init_inclinometer(self) -> Optional[Inclinometer]:
        """Инициализация датчика Inclinometer.
            Вызывается только если НЕ симуляция."""
        # --- Логика для реального Inclinometer ---
        try:
```

```
            inc_config = self.config['sensors']['inclinometer']
            device = inc_config.get('device')
            baudrate = inc_config.get('baudrate', 115200)
            message_header_hex = inc_config.get('message_header_hex')
            if not device or not message_header_hex:
                raise ValueError("Не указаны device или message_header_hex для инклинометра")

            inclinometer = Inclinometer(device=device, baudrate=baudrate,
                                        message_header_hex=message_header_hex)
            self.logger.info(f"Объект Inclinometer создан: {device}@{baudrate}")
            return inclinometer
        except KeyError as e:
            self.logger.error(f"Отсутствует ключ в конфигурации Inclinometer: {e}")
            return None
        except Exception as e:
            self.logger.exception("Ошибка создания объекта Inclinometer")
            return None

    def _init_zmq(self) -> Optional[ZMQHandler]:
        """Инициализация ZMQHandler."""
        try:
            zmq_config = self.config['communication']['zmq']
            port = zmq_config.get('port', 5555)
            identity = zmq_config.get('identity', 'BBBlue')
```

```
            socket_type = zmq_config.get('socket_type', 'DEALER')

            # Определяем хост в зависимости от режима симуляции
            if self.simulate:
                target_host = zmq_config.get('sim_host', 'localhost')
                self.logger.info(f"Режим симуляции: ZMQ будет подключаться к sim_host: {target_host}")
            else:
                target_host = zmq_config.get('host', 'localhost')

            # Используем асинхронный ZMQHandler
            handler = ZMQHandler(
                host=target_host,
                port=port,
                identity=identity,
                socket_type=socket_type,
            )
            return handler
        except KeyError as e:
            self.logger.error(f"Отсутствует ключ в конфигурации ZMQ: {e}")
            return None
        except Exception as e:
            self.logger.exception("Ошибка создания объекта ZMQHandler")
            return None

    def _setup_logging(self):
        """Настройка логирования с использованием RichHandler."""
        try:
```

```
        log_config = self.config.get('logging', {'level': 'INFO', 'file': './bbbsoft.log'})
        log_file = log_config.get('file', './bbbsoft.log')
        log_level_str = log_config.get('level', 'INFO').upper()
        log_level = getattr(logging, log_level_str, logging.INFO)
        log_dir = os.path.dirname(log_file)
        if log_dir and not os.path.exists(log_dir):
            os.makedirs(log_dir, exist_ok=True)

        # Форматтер для файла и консоли
        log_formatter = logging.Formatter("%(asctime)s - %(name)s:%(levelname)s - %(message)s")

        # Обработчик для файла
        file_handler = RotatingFileHandler(
            log_file,
            maxBytes=log_config.get('maxBytes', 10*1024*1024), # 10 MB
            backupCount=log_config.get('backupCount', 5)
        )
        file_handler.setFormatter(log_formatter)

        handler = logging.StreamHandler()
        handler.setFormatter(log_formatter)
        # Отдельный уровень для консоли
        console_log_level_str = log_config.get('console_level', log_level_str).upper()
        console_log_level = getattr(logging, console_log_level_str, log_level)
        handler.setLevel(console_log_level)
```

```
            # Настраиваем базовую конфигурацию
            logging.basicConfig(
                level=log_level,
                # Формат по умолчанию для других логгеров
                format='%(asctime)s - %(name)s:%(levelname)s - %(message)s',
                handlers=[
                    file_handler,
                    handler
                ],
                force=True
            )

            self.logger = logging.getLogger('BBBsoft')
            # Устанавливаем уровень для нашего логгера явно
            self.logger.setLevel(log_level)

            self.logger.info(f'Логирование настроено: Уровень={log_level_str}, Консоль={console_log_level_str}, Файл="{log_file}"')
        except Exception as e:
            # Используем print для вывода, если логгер недоступен
            print(f"CRITICAL: Ошибка настройки логирования: {e}", file=__import__('sys').stderr)
            # Настраиваем простое логирование в stderr как fallback
            logging.basicConfig(level=logging.WARNING,
                                format='%(levelname)s:%(message)s',
                                force=True,
                                handlers=[logging.StreamHandler()])
```

```
            self.logger = logging.getLogger('BBBsoft_fallback')
            self.logger.warning("Используется аварийный режим логирования.")

    async def _adc_reader_task(self):
        """Асинхронная задача для чтения ADC или симуляции данных."""
        task_name = "ADCReaderTask"
        if self.simulate:
            self.logger.info(f"[{task_name}] Запуск задачи генерации симулированных данных ADC...")
        else:
            self.logger.info(f"[{task_name}] Запуск задачи чтения ADC...")
            if not rcpy_available:
                self.logger.error(f"[{task_name}] RCpy или его модуль ADC недоступны, задача чтения не будет запущена.")
                return

        simulation_start_time = time.time()

        while not self._shutdown_event.is_set():
            self.logger.debug(f"[{task_name}] Entering main loop iteration.")
            try:
                # --- Обертка для основного блока ---
                loop = asyncio.get_event_loop()
                start_time = loop.time()
                try:
                    timestamp = time.time() # Общий timestamp

                    if self.simulate:
```

```
                # --- Логика симуляции ADC ---
                sim_time = timestamp - simulation_start_time
                base_value = 2047.5 * (1 + math.sin(2 * math.pi * sim_time / 20.0))
                noise = random.uniform(-50, 50)
                simulated_value = int(max(0, min(4095, base_value + noise)))
                self.adc_data_queue.append((timestamp, simulated_value))
            elif rcpy_available: # Используем флаг доступности rcpy
                # --- Логика чтения реального ADC ---
                try:
                    current_state = rcpy.get_state()
                    if current_state != rcpy.RUNNING:
                        self.logger.warning(f"[{task_name}] Попытка чтения ADC, но RCpy не в состоянии RUNNING ({current_state}). Пропуск чтения.")
                    else:
                        # Используем прямой вызов rcpy.adc
                        # Получаем канал из конфига
                        channel = self.config['sensors']['adc']['channel']
                        # Синхронная функция, но быстрая
                        raw_value = rcpy_adc.get_raw(channel)
```

```
                            self.adc_data_queue.append((timestamp, raw_value))
                            self.logger.debug(f"[{task_name}] ADC read: {raw_value}")
                    except KeyError: # Если канал не задан в конфиге
                        self.logger.error(f"[{task_name}] Канал ADC не найден в конфигурации.")
                        await asyncio.sleep(5.0) # Пауза от спама в лог
                    except AttributeError as e:
                        self.logger.error(f"[{task_name}] Ошибка доступа к rcpy.adc.get_raw: {e}. Проверьте установку rcpy.")
                        await asyncio.sleep(5.0)
                    except Exception as e: # Ловим другие ошибки rcpy
                        self.logger.exception(f"[{task_name}] Ошибка при чтении raw ADC: {e}")
                        await asyncio.sleep(1.0)
                else:
                    # rcpy недоступен, ничего не читаем
                    pass # Не добавляем ничего в очередь

            except Exception as e:
                # Этот блок ловит ошибки внутри основной логики цикла
                log_func = self.logger.exception if not self.simulate else self.logger.error
```

```
                    log_func(f"[{task_name}] Ошибка в основной логике цикла чтения{'' if not self.simulate else симуляции''} ADC: {e}")
                    await asyncio.sleep(1.0) # Добавим паузу и здесь
                finally:
                    # Расчет времени ожидания для поддержания частоты
                    # одинаков( для обоих режимов)
                    # Вынесем в finally, чтобы выполнялось даже при ошибке
                    await self._wait_for_next_cycle(start_time, self.adc_interval_sec, task_name)

            except Exception as e:
                # --- Внешняя обертка для ловли ВСЕХ исключений в цикле ---
                self.logger.exception(f"[{task_name}] КРИТИЧЕСКАЯ НЕОБРАБОТАННАЯ ОШИБКА в главном цикле задачи! {e}")
                # Долгая пауза, чтобы предотвратить быстрое повторение ошибки
                await asyncio.sleep(5.0)

        self.logger.info(f"[{task_name}] Задача завершена.")

    async def _inclinometer_reader_task(self):
        """Асинхронная задача для чтения Inclinometer
           или генерации симулированных данных."""
        task_name = "InclinometerReaderTask"
        if self.simulate:
            self.logger.info(f"[{task_name}] Запуск задачи генерации симулированных данных Inclinometer...")
```

```
        else:
            self.logger.info(f"[{task_name}] Запуск задачи чтения Inclinometer...")
            if not self.inclinometer:
                self.logger.error(f"[{task_name}] Inclinometer не инициализирован, задача чтения не будет запущена.")
                return

        simulation_start_time = time.time()
        inclinometer_interval_sec = 1.0 / self.inclinometer_freq_hz

        while not self._shutdown_event.is_set():
            self.logger.debug(f"[{task_name}] Entering main loop iteration.")
            try:
                loop = asyncio.get_event_loop()
                start_time = loop.time()
                try:
                    timestamp = time.time() # Общий timestamp

                    if self.simulate:
                        # --- Логика симуляции Inclinometer ---
                        sim_time = timestamp - simulation_start_time
                        # Генерируем данные словарем
                        sim_data = {
                            'angle_roll': 10.0 * math.sin(2 * math.pi * sim_time / 30.0) + random.uniform(-0.1, 0.1),
```

```
                        'angle_pitch': 10.0 * math.cos(2 * math.pi * sim_time / 30.0) + random.uniform(-0.1, 0.1),
                        'elevation': 50.0 + sim_time * 0.1 + random.uniform(-0.5, 0.5),
                        'slope': 1.0 + sim_time * 0.01 + random.uniform(-0.05, 0.05),
                        'temp': 25.0 + math.sin(2 * math.pi * sim_time / 60.0)
                    }
                    self.inclinometer_data_queue.append((timestamp, sim_data))
                elif self.inclinometer:
                     # --- Логика чтения реального Inclinometer ---
                    if not self.inclinometer.is_connected():
                        # Попытка переподключения
                        self.logger.warning(f"[{task_name}] Соединение потеряно. Попытка переподключения...")
                        try:
                            await self.inclinometer.connect()
                            if self.inclinometer.is_connected():
                                self.logger.info(f"[{task_name}] Переподключение успешно.")
                            else:
                                self.logger.error(f"[{task_name}] Не удалось переподключиться.")
                                # Пауза перед следующей попыткой
                                await asyncio.sleep(5.0)
```

```
                                continue
                        except Exception as conn_e:
                            self.logger.error(f"[{task_name}] Ошибка при попытке переподключения: {conn_e}")
                            await asyncio.sleep(5.0)
                            continue

                    self.logger.debug(f"[{task_name}] Calling inclinometer.read_data_async()...")
                    data = await self.inclinometer.read_data_async()
                    self.logger.debug(f"[{task_name}] inclinometer.read_data_async() returned: {data is not None}")
                    if data:
                        self.inclinometer_data_queue.append((timestamp,data))
                        self.logger.debug(f"[{task_name}] Inclinometer read: {data}")

            except Exception as e:
                log_func = self.logger.exception if not self.simulate  else self.logger.error
                log_func(f"[{task_name}] Неизвестная ошибка в основной логике цикла чтения{'' if not self.simulate else симуляции''} Inclinometer: {e}")
                # Пауза при неизвестной ошибке
                await asyncio.sleep(5.0)
            finally:
                # Расчет времени ожидания для поддержания частоты
                inclinometer_interval_sec = 1.0 / self.inclinometer_freq_hz
```

```
                    await self._wait_for_next_cycle(start_time, inclinometer_interval_sec, task_name)

            except Exception as e:
                 # --- Внешняя обертка для ловли ВСЕХ исключений в цикле ---
                self.logger.exception(f"[{task_name}] КРИТИЧЕСКАЯ НЕОБРАБОТАННАЯ ОШИБКА в главном цикле задачи! {e}")
                # Долгая пауза
                await asyncio.sleep(5.0)

        self.logger.info(f"[{task_name}] Задача завершена.")
        if not self.simulate and self.inclinometer:
            self.logger.info(f"[{task_name}] Закрываем соединение Inclinometer...")
            await self.inclinometer.close()
            self.logger.info(f"[{task_name}] Соединение Inclinometer закрыто.")

    async def _processing_task(self):
        """Асинхронная задача для обработки данных и отправки."""
        task_name = "ProcessingTask"
        self.logger.info(f"[{task_name}] Запуск задачи обработки данных...")
        if not self.zmq_handler:
            self.logger.error(f"[{task_name}] ZMQHandler не инициализирован, задача обработки не будет запущена.")
            return

        loop = asyncio.get_event_loop()
```

```
        while not self._shutdown_event.is_set():
            # Лог в начале цикла
            self.logger.debug(f"[{task_name}] Entering main loop iteration.")
            try:
                # --- Обертка для основного блока ---
                start_time = loop.time()
                try:
                    current_time = time.time()
                    aggregation_start_time = current_time - self.aggregation_period_sec

                    adc_values_to_process = [val for ts, val in list(self.adc_data_queue) if ts >= aggregation_start_time]
                    inclinometer_dicts_to_process = [d for ts, d in list(self.inclinometer_data_queue) if ts >= aggregation_start_time]

                    avg_adc_raw = None
                    avg_inclinometer_data = None
                    message_to_send = None

                    # --- Усреднение ADC ---
                    if adc_values_to_process:
                        try:
                            avg_adc_raw = statistics.mean(adc_values_to_process)
                            self.logger.debug(f"[{task_name}] ADC Avg Raw ({len(adc_values_to_process)} samples): {avg_adc_raw:.2f}")
                        except statistics.StatisticsError:
```

```
                        self.logger.warning(f"[{task_name}] Недостаточно данных ADC для усреднения ({len(adc_values_to_process)} < 1)")
                    except Exception as e:
                        self.logger.exception(f"[{task_name}] Ошибка усреднения ADC")

                # --- Усреднение Inclinometer ---
                if inclinometer_dicts_to_process:
                    avg_inc_data_temp: Dict[str, float] = {}
                    # Определяем ключи для усреднения динамически
                    if inclinometer_dicts_to_process:
                        keys_to_average = [k for k, v in inclinometer_dicts_to_process[0].items() if isinstance(v, (int, float))]
                    else:
                        keys_to_average = [] # если нет данных

                    valid_samples_count: Dict[str, int] = {k: 0 for k in keys_to_average}

                    for data_dict in inclinometer_dicts_to_process:
                        for key in keys_to_average:
                            if key in data_dict and isinstance(data_dict[key], (int, float)):
                                avg_inc_data_temp[key] = avg_inc_data_temp.get(key, 0) + data_dict[key]
                                valid_samples_count[key] += 1
```

```

                        avg_inclinometer_data = {}
                        for key in keys_to_average:
                            if valid_samples_count[key] > 0:
                                avg_inclinometer_data[key] = avg_inc_data_temp[key] / valid_samples_count[key]
                            else:
                                avg_inclinometer_data[key] = None

                        if avg_inclinometer_data:
                            self.logger.debug(f"[{task_name}] Inclinometer Avg ({len(inclinometer_dicts_to_process)} samples): {avg_inclinome}")
                        elif inclinometer_dicts_to_process:
                            self.logger.warning(f"[{task_name}] Не найдено числовых ключей для усреднения в данных инклинометра.")

                    # --- Формирование и отправка сообщения ---
                    if avg_adc_raw is not None or
                        avg_inclinometer_data is not None:
                        message_to_send = {
                            'timestamp': current_time,
                            'adc_avg_raw': avg_adc_raw,
                            'inclinometer_avg': avg_inclinometer_data,
                            'adc_samples': len(adc_values_to_process),
```

```
                                'inclinometer_samples': len
(inclinometer_dicts_to_process)
                           }

                       if message_to_send:
                           try:
                               success = await self.
zmq_handler.send_async(message_to_send)
                               if success:
                                   self.logger.debug(f"[{
task_name}] Sent: {message_to_send}")
                           except Exception as e:
                               self.logger.exception(f"[{
task_name}] Неожиданная ошибка при вызове send_async
: {e}")

                 except Exception as e:
                     self.logger.exception(f"[{task_name
}] Ошибка в основной логике цикла обработки данных:
{e}")
                     await asyncio.sleep(1.0)
                 finally:
                     await self._wait_for_next_cycle(
start_time, self.processing_interval_sec, task_name)

             except Exception as e:
                 self.logger.exception(f"[{task_name}]
КРИТИЧЕСКАЯ НЕОБРАБОТАННАЯ ОШИБКА в главном цикле
задачи! {e}")
                 # Долгая пауза
                 await asyncio.sleep(5.0)

       self.logger.info(f"[{task_name}] Задача
обработки данных завершена.")
       # Закрываем ZMQ здесь
```

```
        if self.zmq_handler:
            try:
                if hasattr(self.zmq_handler, 'disconnect'):
                    await self.zmq_handler.disconnect()
                else:
                    self.zmq_handler.close()
            except Exception as e:
                self.logger.error(f"[{task_name}] Ошибка закрытия ZMQHandler: {e}")

    async def _wait_for_next_cycle(self, start_time: float,
                                   interval_sec: float, task_name: str):
        """Рассчитывает и выполняет ожидание до следующего цикла задачи."""
        loop = asyncio.get_event_loop()
        elapsed_time = loop.time() - start_time
        wait_time = interval_sec - elapsed_time
        if wait_time > 0:
            await asyncio.sleep(wait_time)
        # Логируем только если опоздание значительное (>10% интервала)
        # и интервал не слишком маленький,
        # чтобы избежать спама при высокой частоте.
        elif interval_sec > 0.001 and wait_time < -interval_sec * 0.1:
             self.logger.warning(f"{task_name} task is lagging: elapsed {elapsed_time:.4f}s > interval {interval_sec:.4f}s")
        # Если wait_time <= 0, но опоздание небольшое, продолжаем без sleep

    async def async_init(self):
```

```
        """Асинхронная часть инициализации
          измерение( частоты или установка по умолчанию)."""
        self.logger.info("Выполнение асинхронной инициализации...")

        if self.simulate:
            default_freq = 5.0
            inc_freq_str = str(self.config.get('sensors', {}) .get('inclinometer', {}) .get('frequency_hz', str(default_freq))).replace(',', '.')
            try:
                self.inclinometer_freq_hz = float(inc_freq_str)
            except ValueError:
                self.logger.warning(f"Не удалось преобразовать inclinometer frequency '{inc_freq_str}' в float для симуляции. Используется {default_freq}.")
                self.inclinometer_freq_hz = default_freq
            if self.inclinometer_freq_hz <= 0:
                self.inclinometer_freq_hz = default_freq
            self.logger.info(f"Режим симуляции: Частота инклинометра установлена в {self.inclinometer_freq_hz:.2f} Гц.")
        else:
            # В обычном режиме измеряем частоту
            measured_freq = await measure_inclinometer_frequency(self.config)
            if measured_freq is not None and measured_freq > 0:
                self.inclinometer_freq_hz = measured_freq
```

```
            else:
                # Используем значение по умолчанию или из конфига,
                # если измерение не удалось
                default_freq = 5.0
                inc_freq_str = str(self.config.get('sensors', {}) .get('inclinometer', {}) .get('frequency_hz', str(default_freq))).replace(',', '.')
                try:
                    self.inclinometer_freq_hz = float(inc_freq_str)
                except ValueError:
                    self.logger.warning(f"Не удалось преобразовать inclinometer frequency '{inc_freq_str}' в float. Используется {default_freq}.")
                    self.inclinometer_freq_hz = default_freq
                if self.inclinometer_freq_hz <= 0:
                    self.inclinometer_freq_hz = default_freq
                self.logger.warning(f"Не удалось измерить частоту инклинометра, используется значение {self.inclinometer_freq_hz:.2f} Гц.")

        # Пересчитываем размер очереди инклинометра и интервал
        self.inclinometer_queue_size = max(1, int(self.inclinometer_freq_hz * self.aggregation_period_sec))
        self.inclinometer_data_queue = deque(maxlen=self.inclinometer_queue_size)
        self.logger.info(f"Inclinometer: частота={self.inclinometer_freq_hz:.2fГц}, очередь={self.inclinometer_queue_size} ({self.aggregation_period_sec})")
```

```
        self.logger.info(f"Processing: частота={self.processing_freq_hzГц}, интервал={self.processing_interval_sec:.3fc}")
        self.logger.info("Асинхронная инициализация завершена.")

    async def run(self):
        """Запуск асинхронных задач системы."""
        self.logger.info("Запуск асинхронных задач...")
        self._shutdown_event.clear()
        self._tasks = []

        # --- Асинхронно подключаемся к ZMQ перед запуском задач ---
        if self.zmq_handler:
            self.logger.debug("Run: Перед вызовом await self.zmq_handler.connect()")
            connect_success = await self.zmq_handler.connect()
            self.logger.debug(f"Run: После вызова await self.zmq_handler.connect(). Успех: {connect_success}")
            if not connect_success:
                self.logger.error("Не удалось подключиться к ZMQ, но запуск продолжается для(отладки).")
        else:
            self.logger.warning("ZMQHandler не инициализирован.")

        # Запускаем задачи чтения и обработки
        self._tasks.append(asyncio.create_task(self._adc_reader_task(), name="ADCReader"))
```

```
        self._tasks.append(asyncio.create_task(self._inclinometer_reader_task(), name="InclinometerReader"))

        self._tasks.append(asyncio.create_task(self._processing_task(), name="ProcessingTask"))

        self.logger.info(f"Запущено {len(self._tasks)} задач. Система работает.")

        # Ожидаем завершения например(, по KeyboardInterrupt)
        await self._shutdown_event.wait()
        self.logger.info("Получено событие остановки. Завершение задач...")

        # Ожидаем завершения всех задач с( таймаутом)
        # Отмена не нужна, тк.. они должны завершиться по _shutdown_event
        done, pending = await asyncio.wait(self._tasks, timeout=5.0)

        if pending:
            self.logger.warning(f"Не все задачи завершились за 5 секунд: {pending}")
            for task in pending:
                task.cancel()
                try:
                    await task
                except asyncio.CancelledError:
                    self.logger.info(f"Задача {task.get_name()} принудительно отменена.")
                except Exception as e:
                    self.logger.error(f"Ошибка при отмене задачи {task.get_name()}: {e}")
```

```
        self.logger.info("Все задачи завершены или отменены.")

    async def stop(self):
        """Инициирует остановку системы."""
        self.logger.info("Инициирована остановка системы...")
        self._shutdown_event.set()

def get_version_info() -> str:
    """Получает информацию о версии, комбинируя SemVer и данные о сборке.

    Использует данные из сгенерированного файла _build_info.py.
    В качестве запасного варианта читаем версию из метаданных пакета.
    """
    semver = _build_info.VERSION

    # Если _build_info не загрузился, пытаемся получить версию из метаданных
    if semver == "0.0.0-unknown-mock":
        try:
            semver = importlib.metadata.version('BBBsoft')
        except importlib.metadata.PackageNotFoundError:
            semver = "0.0.0-unknown"

    commit = _build_info.COMMIT_HASH
    date = _build_info.COMMIT_DATE

    # Формируем строку для вывода
    if commit != "unknown" and date != "unknown":
```

```
        # Убираем часовой пояс и 'T' из даты для краткости, если они есть
        date_short = date.split('+')[0].replace('T', ' ')
        # Убираем секунды для еще большей краткости
        try:
            date_parts = date_short.split(':')
            if len(date_parts) > 2:
                date_short = ':'.join(date_parts[:-1])
        except: pass # Игнорируем ошибки парсинга даты
        return f"{semver} (commit: {commit}, date: {date_short})"
    else:
        return semver # Возвращаем только SemVer, если данных о коммите нет

# --- Точка входа ---
async def main():
    system_version = get_version_info()
    parser = argparse.ArgumentParser(
        description="BBBsoft Measurement System",
        epilog=f"Версия программы: {system_version}"
    )
    parser.add_argument(
        '--simulate',
        action='store_true',
        help='Запуск симуляции без доступа к реальному оборудованию.'
    )
    parser.add_argument(
        '-V', '--version',
        action='version',
        version=f'%(prog)s {system_version}',
        help='Показать версию программы и выйти.'
    )
```

```
    args = parser.parse_args()

    logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s:%(levelname)s - %(message)s')
    bootstrap_logger = logging.getLogger('BBBsoft_bootstrap')
    bootstrap_logger.info(f"Запуск BBBsoft asyncio... Версия: {system_version} Режим( симуляции: ВКЛ{'' if args.simulate else ВЫКЛ''})" )

    config = {}
    config_path = 'config/system_config.yaml'
    try:
        if os.path.exists(config_path):
            with open(config_path, 'r') as f:
                loaded_config = yaml.safe_load(f)
                if loaded_config: config = loaded_config
        else:
            bootstrap_logger.warning(f"Файл конфигурации {config_path} не найден, используются внутренние значения по умолчанию MeasurementSystem.")

    except Exception as e:
        bootstrap_logger.exception(f"Ошибка загрузки конфигурации {config_path}: {e}. Используются внутренние значения по умолчанию.")

    system = None
    try:
        system = MeasurementSystem(config, simulate=args.simulate)
        await system.async_init()
        run_task = asyncio.create_task(system.run(), name="SystemRun")
```

```
        await asyncio.gather(run_task, return_exceptions=True)
    except KeyboardInterrupt:
        bootstrap_logger.info("Получен KeyboardInterrupt. Завершение...")
    except Exception as e:
        bootstrap_logger.exception("Критическая ошибка на верхнем уровне")
    finally:
        if system:
            bootstrap_logger.info("Отправка сигнала остановки системе...")
            await system.stop()

        if not args.simulate:
            try:
                import rcpy
                bootstrap_logger.info("Вызов rcpy.cleanup()...")
                rcpy.cleanup()
                bootstrap_logger.info("rcpy.cleanup() завершен.")
            except ImportError:
                bootstrap_logger.warning("rcpy не найден, пропуск cleanup.")
            except Exception as e:
                bootstrap_logger.error(f"Ошибка при вызове rcpy.cleanup(): {e}")

    bootstrap_logger.info(f"Приложение BBBsoft asyncio Версия(: {system_version}) завершило работу")

if __name__ == "__main__":
    try:
        asyncio.run(main())
```

```
    except Exception as e:
        print(f"FATAL ERROR during asyncio.run: {e}",
  file=sys.stderr)
```

Файл BBBsoft/src/sensors/**adc.py**

```
import os
from typing import Optional

class ADC:
"""Читает сырое значение и масштаб из файловой системы.
  Возвращает
 напряжение в вольтах.
"""
    def __init__(self, channel: str):
        self.channel = channel
        self._initialized = False
    def _setup_adc(self) -> None:
    """Проверка наличия требуемых узлов и путей.
    В тестах `os.path.exists` мокается, достаточно
  вызвать его.
    """
        if not os.path.exists(f"/sys/bus/iio/devices/{
  self.channel}"):
            raise RuntimeError("ADC device path not
  found")
        self._initialized = True
    def read_value(self) -> Optional[float]:
        """Возвращает напряжение (V).
        Производится два последовательных чтения:
        1) сырое значение (counts)
        2) масштаб (V)
        Формула: counts * scale / 1000
        """
```

```
        with open("/tmp/adc_raw", "r", encoding="utf-8") as f_raw:
            counts_str = f_raw.read().strip()
        with open("/tmp/adc_scale", "r", encoding="utf-8") as f_scale:
            scale_str = f_scale.read().strip()
        try:
            counts = float(counts_str)
            scale = float(scale_str)
        except ValueError:
            return None
        return counts * scale / 1000.0

    def close(self) -> None:
    """В текущей конфигурации закрывать ничего не требуется."""
        return
```

Файл BBBsoft/src/sensors/**inclinometer.py**

```
import serial, asyncio, binascii, math
from typing import Dict, Any, Optional, Tuple
import logging, time, threading

logger = logging.getLogger(__name__)

class Inclinometer:
    """Класс для работы с инклинометром по UART с использованием async I/O в executor."""

    DEFAULT_TIMEOUT_SEC = 0.2
```

```
  def __init__(self, device: str, baudrate: int =
115200, message_header_hex: str = "7710FF84",
timeout: float = DEFAULT_TIMEOUT_SEC):
        self.logger = logging.getLogger(__name__)
        self.device = device
        self.baudrate = baudrate
        # Устанавливаем таймаут для синхронных
операций serial
        # Важно: этот таймаут влияет на read(),
readuntil()
        self.timeout = timeout
        self.serial_port: Optional[serial.Serial] =
None
        # Блокировка для синхронизации доступа к порту
        self._lock = threading.Lock()
        self._is_connected = False

        try:
            self.message_header = bytes.fromhex(
message_header_hex.lower())
        except ValueError:
            self.logger.exception(f"Неверный формат HEX
 заголовка: {message_header_hex}")
            raise ValueError("Неверный формат HEX
заголовка")

        self.header_len = len(self.message_header)
        # два BCDзначения- по 4 байта каждое
        self.data_len = 8
        self.crc_len = 1
        self.full_message_length = self.header_len +
self.data_len + self.crc_len
        if self.full_message_length <= self.header_len:
            raise ValueError("Длина полного сообщения
должна быть больше длины заголовка")
```

```
        # Длина заголовка (4 байта)
        self.header_len = len(self.message_header)

        # Предварительно вычисляем длины
        self.header_len_bytes = len(self.message_header
)

        self.logger.info(f"Inclinometer создан: device
={device}, baudrate={baudrate}, header={
message_header_hex}, timeout={self.timeout}s")

  # Метод connect может оставаться async, если
вызывается из async контекста,
  # но сама операция открытия порта синхронная.
  async def connect(self):
        """Устанавливает соединение с UART устройством
операция( синхронная)."""
        # Используем threading.Lock для синхронизации
        with self._lock:
            if self._is_connected and self.serial_port
and self.serial_port.is_open:
                self.logger.debug(f"Уже подключено к {
self.device}")
                return True
            try:
                self.logger.info(f"Подключение к {self.
device}...")
                # Закрываем старое соединение, если
оно есть
                if self.serial_port and self.
serial_port.is_open:
                    self.serial_port.close()

                # Создаем и открываем порт синхронно
```

```
                self.serial_port = serial.Serial(
                    port=self.device,
                    baudrate=self.baudrate,
                    # Таймаут для read операций
                    timeout=self.timeout
                )
                # Проверка, открылся ли порт
                if self.serial_port.is_open:
                    self._is_connected = True
                    # Очищаем буфер при подключении
                    self.serial_port.reset_input_buffer()
                    self.logger.info(f"Успешно подключено к {self.device}")
                    return True
                else:
                    self.logger.error(f"Не удалось открыть порт {self.device}, is_open=False")
                    self._is_connected = False
                    self.serial_port = None
                    return False
            except serial.SerialException as e:
                self.logger.error(f"Ошибка serial при подключении к {self.device}: {e}")
                self._is_connected = False
                self.serial_port = None
                return False
            except Exception as e:
                self.logger.exception(f"Неожиданная ошибка при подключении к {self.device}")
                self._is_connected = False
                self.serial_port = None
                return False

    def is_connected(self) -> bool:
```

```
        """Возвращает статус подключения."""
        with self._lock:
            return self._is_connected and self.serial_port is not None and self.serial_port.is_open

    # Метод close может оставаться async, если вызывается из async контекста,
    # но сама операция закрытия порта синхронная.
    async def close(self):
        """Закрывает соединение операция( синхронная)."""
        with self._lock:
            if not self._is_connected or not self.serial_port:
                self.logger.debug(f"Соединение с {self.device} уже было закрыто или не установлено.")
                return
            try:
                self.logger.info(f"Закрытие соединения с {self.device}...")
                if self.serial_port and self.serial_port.is_open:
                    self.serial_port.close()
                self._is_connected = False
                # Сбрасываем объект порта
                self.serial_port = None
                self.logger.info(f"Соединение с {self.device} закрыто.")
            except Exception as e:
                self.logger.exception(f"Ошибка при закрытии соединения с {self.device}")
                # Все равно сбрасываем флаги и объект
                self._is_connected = False
                self.serial_port = None
```

```
    # --- Асинхронный метод- обертка ---
    async def read_data_async(self) -> Optional[Dict[str, float]]:
        """Асинхронно читает и разбирает сообщение от инклинометра.

        Returns:
            Optional[Dict[str, float]]: Словарь с углами наклона или None при ошибке
        """
        if not self._is_connected or not self.serial_port:
            try:
                if not await self.connect():
                    self.logger.error("Failed to reconnect to inclinometer")
                    return None
            except Exception as e:
                self.logger.error(f"Error reconnecting to inclinometer: {e}")
                return None

        try:
            loop = asyncio.get_running_loop()
            result = await loop.run_in_executor(None, self._sync_read_parse_message)
            if result is None:
                self.logger.warning("Failed to read valid data from inclinometer")
            return result
        except serial.SerialException as e:
            self.logger.error(f"Serial communication error: {e}")
            try:
                # Закрываем порт при ошибке
```

```python
                await self.close()
            except Exception:
                pass
            return None
        except Exception as e:
            self.logger.error(f"Unexpected error reading inclinometer data: {e}")
            return None

    # --- Синхронная функция для выполнения в executor ---
    def _sync_read_parse_message(self) -> Optional[Dict[str, float]]:
        """Синхронное чтение и разбор сообщения.

        Returns:
            Dict с углами наклона или None в случае ошибки
        """
        if not self._is_connected or not self.serial_port:
            logger.warning("Порт закрыт")
            return None

        try:
            with self._lock:
                # Ищем заголовок
                header = bytes()
                while len(header) < len(self.message_header):
                    # Явно указываем размер чтения
                    byte = self.serial_port.read(1)
                    if not byte:
                        logger.warning("Таймаут при чтении заголовка")
```

```
                    return None
                header += byte
                # Если накопленный заголовок не совпадает с началом message_header,
                # отбрасываем первый байт и продолжаем поиск
                if not self.message_header.startswith(header):
                    header = header[1:] if len(header) > 1 else bytes()

            if header != self.message_header:
                logger.warning("Неверный заголовок сообщения")
                return None

            # Читаем данные (8 байт)
            payload = bytes()
            for _ in range(8):
                byte = self.serial_port.read(1)
                if not byte:
                    logger.warning("Таймаут при чтении данных")
                    return None
                payload += byte

            # Пропускаем байт контрольной суммы
            self.serial_port.read(1)

            # Разбираем данные
            try:
                angle1 = float(self._bcd_decode(payload[0:4]))
                angle2 = float(self._bcd_decode(payload[4:8]))
```

```
                except ValueError as e:
                    logger.warning(f"Ошибка декодирования BCD: {e}")
                    return None

                return {
                    'angle_roll': angle1,
                    'angle_pitch': angle2
                }

        except serial.SerialTimeoutException:
            logger.warning("Таймаут при чтении данных")
            return None
        except Exception as e:
            logger.error(f"Ошибка при чтении данных: {e}")
            return None

    def _parse_message(self, message: bytes) -> Dict[str, Any]:
        angle_roll_raw = int.from_bytes(message[4:6], byteorder='big', signed=True)
        angle_pitch_raw = int.from_bytes(message[6:8], byteorder='big', signed=True)
        elevation_raw = int.from_bytes(message[8:9], byteorder='big', signed=True)
        slope_raw = int.from_bytes(message[9:10], byteorder='big', signed=False)
        angle_roll = angle_roll_raw * 0.01
        angle_pitch = angle_pitch_raw * 0.01
        elevation = elevation_raw * 0.5
        slope = slope_raw * 0.1
        data = {
            'angle_roll': angle_roll,
            'angle_pitch': angle_pitch,
```

```
            'elevation': elevation,
            'slope': slope,
        }
        return data

    def _bcd_decode(self, oBytes: bytes) -> str:
        """Декодирует 4- байтовое BCD- значение в строку с плавающей точкой."""
        # первый байт: знак в BCD 0x10 означает отрицательное
        sign = '-' if binascii.hexlify(oBytes[0:1]).decode() == '10' else ''
        int_part = binascii.hexlify(oBytes[1:2]).decode()
        frac1 = binascii.hexlify(oBytes[2:3]).decode()
        frac2 = binascii.hexlify(oBytes[3:4]).decode()
        return f"{sign}{int_part}.{frac1}{frac2}"

    # Синхронный метод для тестов и legacy API
    def _setup_port(self):
        """Инициализация serial- порта через pyserial."""
        try:
            self.serial_port = serial.Serial(
                port=self.device,
                baudrate=self.baudrate,
                timeout=self.timeout
            )
        except serial.SerialException as e:
            raise RuntimeError(f"Не удалось открыть порт {self.device}: {e}")
```

Файл BBBsoft/src/communication/**zmq_handler.py**

```python
import asyncio, zmq
import zmq.asyncio as zmq_async
import os, time, logging, json
from typing import Any, Dict, Optional
from datetime import datetime

logger = logging.getLogger(__name__)

class ZMQHandler:
    """Асинхронный обработчик для отправки данных по ZeroMQ.

    Поддерживает обратную совместимость с конструктором вида ZMQHandler(config_dict).
    """
    def __init__(
        self,
        host: str = 'localhost',
        port: int = 5555,
        identity: str = "BBBsoft",
        socket_type: str = 'DEALER',
        *,
        save_local: bool = False,
        local_path: Optional[str] = None,
        config: Optional[Dict[str, Any]] = None,
    ):
        # Поддержка конструктора с config dict
        if isinstance(host, dict) and config is None:
            config = host  # type: ignore[assignment]
        if config is not None:
            self.protocol = config.get('protocol', 'tcp')
            self.host = config.get('host', 'localhost')
            self.port = int(config.get('port', 5555))
```

```
            self.identity = config.get('identity', identity)
            self.socket_type = config.get('socket_type', socket_type)
        else:
            self.protocol = 'tcp'
            self.host = host
            self.port = port
            self.identity = identity
            self.socket_type = socket_type

        # Путь и локальное сохранение
        self.save_local = save_local
        self.local_path = local_path or os.path.join(os.getcwd(), 'logs')

        # Тип сокета
        self.socket_type_enum = getattr(zmq, socket_type.upper(), zmq.DEALER)
        self.context: Optional[zmq_async.Context] = None
        self.socket: Optional[zmq_async.Socket] = None
        self._connection_lock = asyncio.Lock()
        self._is_connected = False

        logger.info(f"Async ZMQHandler инициализирован для {self.url} Тип(: {socket_type.upper()})")

    @property
    def url(self) -> str:
        return f"{self.protocol}://{self.host}:{self.port}"

    async def connect(self) -> bool:
```

```
        """Устанавливает асинхронное соединение ZeroMQ."""
        async with self._connection_lock:
            if self._is_connected and self.socket and not self.socket.closed:
                logger.debug("ZMQ уже подключен.")
                return True

            try:
                # Закрываем старое соединение, если оно есть синхронно()
                self.close()

                logger.info(f"ZMQ connect: Попытка подключения к {self.url}...")
                logger.debug(f"ZMQ connect: Перед созданием Context...")
                self.context = zmq_async.Context.instance()
                logger.debug(f"ZMQ connect: Context создан: {self.context}")

                logger.debug(f"ZMQ connect: Перед созданием Socket типа {self.socket_type_enum}...")
                self.socket = self.context.socket(self.socket_type_enum)
                logger.debug(f"ZMQ connect: Socket создан: {self.socket}")

                logger.debug(f"ZMQ connect: Перед установкой IDENTITY: {self.identity}")
                self.socket.setsockopt_string(zmq.IDENTITY, self.identity)

```

```
                logger.debug(f"ZMQ connect: Перед установкой LINGER: 0")
                self.socket.setsockopt(zmq.LINGER, 0)

                logger.debug(f"ZMQ connect: Перед вызовом self.socket.connect()...")
                self.socket.connect(self.url)
                logger.debug(f"ZMQ connect: Вызов self.socket.connect() завершен.")

                self._is_connected = True
                logger.info("ZMQ подключение успешно.")
                logger.debug(f"ZMQ connect: Установлен флаг _is_connected=True")
                return True
            except Exception as e:
                logger.exception(f"Ошибка подключения ZMQ: {e}")
                self.close()  # Попытка закрыть ресурсы при ошибке
                return False

    def is_connected(self) -> bool:
        """Проверяет статус соединения."""
        # Проверяем флаг и наличие сокета
        return self._is_connected and self.socket is not None and not self.socket.closed

    def send_data(self, sensor_type: str, data: Dict[str, Any]) -> bool:
        """
        Отправка данных через ZeroMQ

        Args:
```

```
            sensor_type: Тип датчика (INC, Odometer, DSHK, GNSS, INS)
            data: Данные для отправки

        Returns:
            bool: Успешность отправки
        """
        try:
            # Добавляем метку времени
            data['time'] = time.time()

            # Формируем полный пакет
            message = {
                "identity": sensor_type,
                sensor_type: data
            }

            # Отправляем через ZMQ
            self.socket.send_json(message)
            return True
        except Exception as e:
            print(f"Ошибка отправки данных (send_data): {e}")
            return False

    async def send_async(self, message: Dict[str, Any]) -> bool:
        """Асинхронная отправка сообщения словаря() через ZeroMQ."""
        if not self.is_connected():
            logger.warning("Попытка отправки ZMQ при отсутствии соединения. Попытка подключения...")
            if not await self.connect():
                logger.error("Не удалось подключиться для отправки ZMQ.")
```

```
                return False

        # Убедимся, что сокет существует после возможного реконнекта
        if not self.socket: return False

        try:
            await self.socket.send_json(message)
            return True
        except zmq.ZMQError as e: # <-- Ловим базовый zmq.ZMQError
            logger.error(f"Ошибка ZMQ при отправке: {e} Код(: {e.errno})")
            if e.errno == zmq.EFSM:
                logger.warning("Попытка переподключения ZMQ изза- ошибки состояния...")
                self.close() # Закрываем перед реконнектом
            # Возвращаем False, чтобы вызывающий код знал об ошибке
            return False
        except Exception as e:
            logger.exception(f"Неожиданная ошибка при отправке ZMQ: {e}")
            self.close()
            return False

    async def disconnect(self):
        """Асинхронная обертка для закрытия."""
        self.close()

    def close(self):
        """Синхронное закрытие сокета и контекста ZeroMQ."""
        if self.socket and not self.socket.closed:
```

```
            logger.info("Закрытие ZMQ сокета...")
            try:
                self.socket.close()
            except Exception as e:
                logger.error(f"Ошибка при закрытии ZMQ
сокета: {e}")
        self.socket = None
        self._is_connected = False
        self.context = None
        logger.debug("ZMQ ресурсы освобождены сокет(
закрыт).")

    async def send_data(self, data: Dict[str, Any]) ->
None:
    """Асинхронная отправка с формированием конверта.

    Под тесты ожидается структура {"timestamp": <float
>, "data": <dict>}.
    Исключения не подавляются пусть( пробрасываются
для тестов).
    """
        if not self.is_connected():
            await self.connect()
        if not self.socket:
            raise RuntimeError("ZMQ socket is not
available")
        envelope = {"timestamp": time.time(), "data":
data}
        await self.socket.send_json(envelope)

    # Вспомогательные методы для локального сохранения
    def _ensure_log_directory(self) -> None:
        if not os.path.exists(self.local_path):
            os.makedirs(self.local_path)
```

```python
    def _save_data(self, data: Dict[str, Any]) -> None:
        if not self.save_local:
            return
        self._ensure_log_directory()
        file_path = os.path.join(self.local_path, "zmq_data.json")
        try:
            with open(file_path, "w", encoding="utf-8") as f:
                f.write(json.dumps(data, ensure_ascii=False))
        except Exception as e:
            logger.error(f"Ошибка локального сохранения данных: {e}")
```

Файл конфигурации BBBsoft/config/**system_config.yaml**

```yaml
# Конфигурация модуля BBBsoft

# Настройки логирования
logging:
  level: INFO
  file: ./logs/bbbsoft.log
  max_size: 10485760  # 10MB
  backup_count: 5

# Настройки датчиков
sensors:
  # ДШК
  adc:
    # Канал АЦП. Частота опроса задается параметром frequency_hz.
    channel: 0                    # Номер канала АЦП (0-7)
```

```
    frequency_hz: 10                    # Частота опроса АЦП
  # Инклинометр
  inclinometer:
    device: /dev/ttyS5                  # Порт подключения
    baudrate: 115200                    # Скорость соединения
    message_header_hex: "7710ff84"   # Шестнадцатеричный заголовок сообщения

# Настройки коммуникации
communication:
  zmq:
    host: 192.168.1.184   # IPадрес- основного сервера ZMQ
    sim_host: localhost   # IPадрес- сервера ZMQ для режима симуляции
    port: 5555            # Порт ZMQ
    identity: BBBlue      # Идентификатор клиента (BeagleBone Blue)
    socket_type: DEALER   # Тип сокета ZMQ

# Настройки основного цикла
processing:
  frequency_hz: 1,      # Периодичность усреднения и отправки данных Гц()
  aggregation_period_sec: 1   # Длительность окна усреднения данных сек()
```

Модуль пользовательского интерфейса и управления

Файл HMI-Tah/src/**gui.py**

```
from pathlib import Path
import threading
from tkinter import Tk, Canvas, Text, Button,
  PhotoImage, Toplevel, Label, END
import serial
from pygeocom import PyGeoCom, LockInStatus,
  MeasurementProgram, TMCInclinationMode
from matplotlib.backends.backend_tkagg import
  FigureCanvasTkAgg
from matplotlib.figure import Figure
from matplotlib.patches import Arc
import xml.etree.ElementTree as ET
import time
import zmq
import json
import csv
import numpy as np
import sys
from pyclothoids import SolveG2

flag_start_visual = 0
flag_TAH_connection = 0
flag_connect_ok = 0
flag_retry = 1
flag_prism_lock = 0
flag_stop = 0
flag_app_end = 0
flag_established_connection = 0
flag_connect_lost = 0
```

```
next_status = 'None'
way_status = 'None'
next_coor = (0, 0)
offset_prism = [0.866179, 0.039005, 0.627931]
try:
    config = open('./resources/config.txt','r')
    data = json.load(config)

    com_port = data["Bluetooth_COM"]
    landxml_name = data["LandXML_Name"]
    result_name = data["Calculation_Name"]
    tah_data = data["Raw_tacheometer_data_Name"]
    beagle_data = data["Raw_sensor_data_Name"]
    m_DSHK = data["DSHK_scale"]
    DSHK_y_offset = data["DSHK_offset"]
    INC_roll_offset = data["INC_roll_offset"]
    INC_pitch_offset = data["INC_pitch_offset"]

    config.close()

except:
    config = open('./resources/config.txt','w')

    data = {"Bluetooth_COM": "COM6",
            "LandXML_Name": "1",
            "Calculation_Name": "result",
            "Raw_tacheometer_data_Name": "raw_data",
            "Raw_sensor_data_Name": "raw_Beagle_data",
            "DSHK_scale": 0,
            "DSHK_offset": 0,
            "INC_roll_offset": 0,
            "INC_pitch_offset":0}
    json.dump(data, config)

    config.close()
```

```
    com_port = data["Bluetooth_COM"]
    landxml_name = data["LandXML_Name"]
    result_name = data["Calculation_Name"]
    tah_data = data["Raw_tacheometer_data_Name"]
    beagle_data = data["Raw_sensor_data_Name"]
    m_DSHK = data["DSHK_scale"]
    DSHK_y_offset = data["DSHK_offset"]
    INC_roll_offset = data["INC_roll_offset"]
    INC_pitch_offset = data["INC_pitch_offset"]


lineList = []
spiralList = []
curveList = []

window_connection = None
lock_status = None

# Счетчик измерений
counter = 0

# Данные от датчиков
TAH_measurements = [[0,0,0]]
INC_data = [0, 0]
DSHK_data = [0]
rail_height = 0

# Расчеты сравнения Landxml
landxml_status = "off_track"
center_shift = 0
height_shift = 0

# Результаты расчета
L_R = [0, 0, 0]
```

```
C_R = [0, 0, 0]
R_R = [0, 0, 0]
path_W = 0

# Инициализация первого измерения тахеометра
TAH_last_x = 0
TAH_last_y = 0

# Вывод консоли
class ConsoleRedirector:
    def __init__(self, text_widget):
        self.text_widget = text_widget

    def write(self, message):
        self.text_widget.insert(END, message)
        self.text_widget.see(END)

    def flush(self):
        pass

# Подключение к BeagleBoneBlue для получения данных от инклинометра и датчика ширины колеи
def Beagle_connection():
    global counter, INC_data, DSHK_data, landxml_status, center_shift, way_status, next_status, next_coor, height_shift
    raw_B = ["DHSK", "INC_roll", "INC_pitch"]
    while True:

        if flag_app_end == 1:
            break


        i, d_message = server_socket.recv_multipart()
        message = json.loads(d_message.decode())
```

```

        try:

            if message["inclinometer_avg"] is not None:
                INC_data = [message["inclinometer_avg"]["angle_roll"], message["inclinometer_avg"]["angle_pitch"]]
            else:
                print("Отсутствуют данные от инклинометра")

            if message["adc_avg_raw"] is not None:
                DSHK_data = [message["adc_avg_raw"]]
            else:
                print("Отсутвуют данные от датчика ширины колеи")


            # Запись сырых данных
            if flag_start_visual == 1 and flag_connect_ok == 1:
                f_raw_B = open(f'{beagle_data}.csv','a')

                csvwriter_raw = csv.writer(f_raw_B)
                csvwriter_raw.writerow(raw_B)
                raw_B = [DSHK_data[0], INC_data[0], INC_data[1]]

        except:

            landxml_status = message["status"]
            center_shift = float(message["shifts"])
            height_shift = float(message["height_dif"])
            try:
```

```
                way_status = message["current_loc"]['type']
                next_status = message["next_loc"]['type']

                next_coor = message["next_loc"]["start"]

            except:
                pass

            message_out = {"C_R_x": C_R[0], "C_R_y": C_R[1], "C_R_z": C_R[2], "H": rail_height/2}
            server_socket.send_multipart([i, json.dumps(message_out).encode()])
        time.sleep(0.05)



# Расчет параметров рельса и их запись
def calcute_params():
    global counter, TAH_last_x, TAH_last_y, L_R, C_R, R_R, path_W, rail_height
    raw = ["Tah_x", "Tah_y", "Tah_z"]
    result = ["Number", "Left_rail_x", "Left_rail_y", "Left_rail_z", "Center_x", "Center_y", "Center_z", "Right_rail_x", "Right_rail_y", "Right_rail_z", "shift", "height" ]

    while True:

        if flag_start_visual == 1 and flag_connect_ok == 1:
```

```
        f_result = open(f'{result_name}.csv','a')
        f_raw = open(f'{tah_data}.csv','a')

        if (TAH_measurements[0][0] - TAH_last_x !=0) and (TAH_measurements[0][0] != 0):
            counter += 1

            try:
                TAH_delta_x = TAH_measurements[0][0] - TAH_last_x
                TAH_delta_y = TAH_measurements[0][1] - TAH_last_y
            except:
                TAH_delta_x = 0
                TAH_delta_y = 0


            if abs(TAH_delta_x) > 0.05 and abs(TAH_last_y) > 0.05:
                psi = np.arctan2(TAH_delta_y, TAH_delta_x)

            # Расчет угла крена
            roll = - np.radians(INC_data[0]-float(INC_roll_offset))

            # Расчет ширины колеи
            path_W = (DSHK_y_offset + m_DSHK*DSHK_data[0])

            try:
                delta_L_x = (offset_prism[0] * np.cos(roll) + offset_prism[2] * np.sin(roll)) * np.cos(np.pi/2 - psi) + offset_prism[1] * np.cos(psi)
```

```
                delta_L_y = - offset_prism[0] * np.sin(np.pi/2 - psi) + offset_prism[1] * np.sin(psi)
                delta_L_z = - offset_prism[0] * np.sin(roll) + offset_prism[2] * np.cos(roll)

                delta_L = np.array([delta_L_x, delta_L_y, delta_L_z])

                L_R = np.array(TAH_measurements[0]) - delta_L


                delta_R_x = (- (path_W - offset_prism[0])* np.cos(roll) + offset_prism[2] * np.sin(roll)) * np.cos(np.pi/2 - psi) + offset_prism[1] * np.cos(psi)
                delta_R_y = (path_W - offset_prism[0]) * np.sin(np.pi/2 - psi) + offset_prism[1] * np.sin(psi)
                delta_R_z = (path_W - offset_prism[0]) * np.sin(roll) + offset_prism[2] * np.cos(roll)

                delta_R = np.array([delta_R_x, delta_R_y, delta_R_z])

                R_R = np.array(TAH_measurements[0]) - delta_R

                delta_C_x = (- (path_W/2 - offset_prism[0])* np.cos(roll) + offset_prism[2] * np.sin(roll)) * np.cos(np.pi/2 - psi) + offset_prism[1] * np.cos(psi)
                delta_C_y = (path_W/2 - offset_prism[0]) * np.sin(np.pi/2 - psi) + offset_prism[1] * np.sin(psi)
```

```
                    delta_C_z = (path_W/2 - offset_prism[0]) * np.sin(roll) + offset_prism[2] * np.cos(roll)
                    delta_C = np.array([delta_C_x, delta_C_y, delta_C_z])

                    C_R = np.array(TAH_measurements[0]) - delta_C

                    rail_height = R_R[2] - L_R[2]

                    csvwriter_raw = csv.writer(f_raw)
                    csvwriter_raw.writerow(raw)
                    raw = [TAH_measurements[0][0], TAH_measurements[0][1], TAH_measurements[0][2]]

                    csvwriter = csv.writer(f_result)
                    csvwriter.writerow(result)
                    result = [counter, round(L_R[0],4), round(L_R[1],4), round(L_R[2],4), round(C_R[0],4), round(C_R[1],4), round(C_R[2],4), round(R_R[0],4), round(R_R[1],4), round(R_R[2],4), center_shift, height_shift]
                except:
                    print("Для начала измерений начните движение")

                TAH_last_x = TAH_measurements[0][0]
                TAH_last_y = TAH_measurements[0][1]

        if flag_app_end == 1:
            break

        time.sleep(0.05)
```

```
# Подключение к тахеометру
def TAH_connection():
    def serial_connection(port, baudrate, cooldown = 9):
        start_time = time.time()
        while True:
            try:
                ser = serial.Serial(port=port, baudrate=baudrate, timeout=10)
                return ser, 1

            except:
                pass

            if time.time() - start_time > cooldown:
                print("Ошибка при подключении к Тахеометру")
                break
            time.sleep(1)
        return None, 0

    global flag_TAH_connection, flag_start_visual, flag_retry, flag_prism_lock, TAH_measurements, flag_connect_ok, flag_established_connection, TAH_measur, flag_connect_lost
    while True:

        if flag_start_visual == 1 and flag_retry == 1:
            try:
                geo.check_power()
            except:
                tah_ser, flag_TAH_connection = serial_connection(f'{com_port}', 19200)

            flag_retry = 0
```

```
        if flag_TAH_connection == 1:
            print("Соединение с тахеометром -
Установлено")

            geo = PyGeoCom(tah_ser, debug = False)
            try:
                geo.search(0.1, 0.1)
                geo.user_lock_state_on()
                geo.lock_in()
                geo.set_measurement_program(
MeasurementProgram.CONT_REF_STANDARD)
                if geo.get_motor_lock_status()==
LockInStatus.LOCKED_OUT:
                    flag_prism_lock = 0
                else:
                    flag_prism_lock = 1
            except:
                print("Наведите тахеометр на
Призму")


    if flag_connect_ok == 1:
        try:
            if geo.get_motor_lock_status()==
LockInStatus.LOCKED_IN:

                flag_prism_lock = 1
                TAH_measurements = geo.
get_coordinate(TMCInclinationMode.AUTOMATIC)

                time.sleep(1)
            elif geo.get_motor_lock_status()==
LockInStatus.LOCKED_OUT:
                flag_prism_lock = 0
```

```
                    print("Призма потеряна")
                    time.sleep(1)
                    try:
                        geo.search(0.1, 0.1)
                        geo.lock_in()
                    except:
                        print("Ошибка поиска призмы")
            except:
                print("Потеряно соединение с тахеометром")
                tah_ser.close()
                flag_TAH_connection = 0
                flag_connect_ok = 0
                flag_prism_lock = 0
                flag_established_connection = 0
                flag_connect_lost = 1
                time.sleep(1)

        if flag_app_end == 1:
            try:
                tah_ser.close()
            finally:
                break

        if flag_stop == 1:
            try:
                if flag_prism_lock == 1:
                    geo.user_lock_state_off()
            finally:
                flag_connect_ok = 0

                time.sleep(1)

        time.sleep(0.05)
```

```
# Открытие сокета сервера
context = zmq.Context()
server_socket = context.socket(zmq.ROUTER)
server_socket.bind("tcp://*:5555")

# Поток подключения к тахеометру
thread1 = threading.Thread(target=TAH_connection)
thread1.start()

# Поток подключения к BeagleBone
thread2 = threading.Thread(target=Beagle_connection)
thread2.start()

# Поток расчета
thread3 = threading.Thread(target=calcute_params)
thread3.start()



OUTPUT_PATH = Path(__file__).parent
ASSETS_PATH = OUTPUT_PATH / Path(r"../resources/assets/
    frame0")

def relative_to_assets(path: str) -> Path:
    return ASSETS_PATH / Path(path)


#Ввод------------ значений---------------------
def open_popup():
    top = Toplevel(window)
    top.configure(bg = "#141831")
    top.geometry("450x250")
    top.title("Ввод параметров")
    top.grab_set()
```

```
  canvas_popup = Canvas(top, bg = "#151831", height = 250, width = 450, bd = 0, highlightthickness = 0, relief = "ridge")

  canvas_popup.place(x = 0, y = 0)
  input_image_1 = PhotoImage(file=relative_to_assets("input_image_1.png"))
  input_image1 = canvas_popup.create_image(150.0, 16.0, image=input_image_1)

  input_image_2 = PhotoImage(file=relative_to_assets("input_image_2.png"))
  input_image2 = canvas_popup.create_image(150.0, 46.0, image=input_image_2)

  input_image_3 = PhotoImage(file=relative_to_assets("input_image_3.png"))
  input_image3 = canvas_popup.create_image(150.0, 76.0, image=input_image_3)

  input_image_4 = PhotoImage(file=relative_to_assets("input_image_4.png"))
  input_image4 = canvas_popup.create_image(150.0, 106.0, image=input_image_4)

  input_image_5 = PhotoImage(file=relative_to_assets("input_image_5.png"))
  input_image5 = canvas_popup.create_image(150.0, 136.0, image=input_image_5)

  input_image_6 = PhotoImage(file=relative_to_assets("input_image_6.png"))
  input_image6 = canvas_popup.create_image(150.0, 166.0, image=input_image_6)
```

```
def change_start():

    def countdown(name, n):
        global flag_established_connection, flag_connect_lost, lock_status

        def connect_ok():
            global flag_connect_ok
            flag_connect_ok = 1
            button_start.config(state='disabled')
            window_connection.destroy()

        def retry(name):
            global flag_retry, lock_status
            flag_retry = 1
            countdown(name, 10)
            retry_button.destroy()
            close_button.destroy()
            try:
                lock_status.destroy()
            except:
                pass


        if n > 0:

            if flag_TAH_connection == 0:
                name.config(text=f"{n} сек...")
                window_connection.after(1000, countdown, name, n-1)
            elif flag_TAH_connection == 1 and flag_established_connection == 0:
```

```
                    name.config(text="Подключение
установлено")
                    flag_connect_lost = 0
                    flag_established_connection = 1
                    window_connection.after(1000, countdown
, name, n-1)
            elif flag_TAH_connection == 1 and
flag_established_connection == 1:
                    start_time = time.time()
                    while True:
                        if flag_prism_lock == 1:

                                lock_status =  Label(
window_connection, anchor="center", text="Цель
захвачена", font=("Almarai Bold", 16 * -1))
                                lock_status.pack()
                                connect_ok_button = Button(
window_connection, text="Начать измерение", anchor="
center", font=("Almarai Bold", 16 * -1), command = lambd
:connect_ok())
                                connect_ok_button.pack()
                                break
                        if time.time() - start_time > 5:
                                lock_status =  Label(
window_connection, anchor="center", text="Цель не
найдена", font=("Almarai Bold", 16 * -1))
                                lock_status.pack()
                                retry_button = Button(
window_connection, text="Повтор", anchor="center",
font=("Almarai Bold", 16 * -1), command = lambda:retry
(name))
                                retry_button.pack()
                                close_button = Button(
window_connection, text="Отмена", anchor="center",
```

```
font=("Almarai Bold", 16 * -1), command=window_connection
.destroy)
                        close_button.pack()
                        break
      else:
          name.config(text="Ошибка при подключении")

          close_button = Button(window_connection,
text="Ок", anchor="center", font=("Almarai Bold", 16
 * -1), command=window_connection.destroy)
          close_button.pack()

          retry_button = Button(window_connection,
text="Переподключение", anchor="center", font=("
Almarai Bold", 16 * -1), command = lambda:retry(name)
)
          retry_button.pack()

  global flag_start_visual, flag_stop, flag_retry,
flag_connect_lost, window_connection

  if window_connection is not None and
window_connection.winfo_exists():
      return

  flag_start_visual = 1
  flag_stop = 0
  flag_retry = 1

  popup_width = 250
  popup_height = 200
  screen_width = 1280
  screen_height = 720
  x = (screen_width // 2) - (popup_width // 2)
  y = (screen_height // 2) - (popup_height // 2)
```

```
    window_connection = Toplevel(window)
    window_connection.geometry(f"{popup_width}x{popup_height}+{x}+{y}")
    if flag_connect_lost == 0:
        window_connection.title("Инициализация...")
    elif flag_connect_lost == 1:
        window_connection.title("Переподключение...")
    window_connection.grab_set()

    init_status = Label(window_connection, anchor="center", text="Подключение к тахеометру", font=("Almarai Bold", 16 * -1))
    init_status.pack()
    if flag_connect_lost == 1:
        init_status.config(text="Переподключение к тахеометру")
        button_start.config(state='normal')
    init_time =  Label(window_connection, anchor="center", text="Подключение установлено", font=("Almarai Bold", 16 * -1))
    init_time.pack()

    countdown(init_time, 10)

    window_connection.resizable(False, False)


def change_stop():
    global flag_start_visual, flag_start_write, flag_stop, flag_app_end
    button_start.config(state='active')
    flag_start_visual = 0
    flag_start_write = 0
    if flag_stop == 1:
```

```
        flag_app_end = 1
        window.destroy()
    flag_stop = 1

# Основное окно

window = Tk()
window.title("Параметры измерительного комплекса")
window.geometry("1280x720")
window.configure(bg = "#004689")

canvas = Canvas(window, bg = "#004689", height = 720, width = 1280, bd = 0, highlightthickness = 0, relief = "ridge")

canvas.place(x = 0, y = 0)

# Чтение LANDXML
try:
    track_segments = []
    tree = ET.parse(f'./resources/{landxml_name}.xml')
    root = tree.getroot()

    namespace_uri = root.tag[1:].split("}")[0]
    if namespace_uri == 'http://www.landxml.org/schema/LandXML-1.2':
        ns = '{http://www.landxml.org/schema/LandXML-1.2}'
        version = "1.2"
        print(version)
    elif namespace_uri == 'http://www.landxml.org/schema/LandXML-1.1':
        ns = '{http://www.landxml.org/schema/LandXML-1.1}'
        version = "1.1"
```

```
        print(version)
    else:
        print("Неподдерживающийся тип LandXML:", namespace_uri)

    for alignment in root.findall(ns + 'Alignments/' + ns + 'Alignment'):

        coordGeom = alignment.find(ns + 'CoordGeom')

        if coordGeom is not None:

            for obj in coordGeom:
                if obj.tag == ns + 'Line':
                    line_data = [
                        [*map(float, obj.find(ns + 'Start').text.split())],
                        [*map(float, obj.find(ns + 'End').text.split())]
                    ]
                    lineList.append(line_data)

                elif obj.tag == ns + 'Spiral':
                    spiral_data = [
                        [*map(float, obj.find(ns + 'Start').text.split())],
                        [*map(float, obj.find(ns + 'PI').text.split())],
                        [*map(float, obj.find(ns + 'End').text.split())],
                        (obj.attrib['rot']),
                        (obj.attrib['radiusStart']),
                        (obj.attrib['radiusEnd'])
                    ]
                    spiralList.append(spiral_data)
```

```
            elif obj.tag == ns + 'Curve':
                curve_data = [
                    [*map(float, obj.find(ns + '
Start').text.split())],
                    [*map(float, obj.find(ns + 'End
').text.split())],
                    [*map(float, obj.find(ns + '
Center').text.split())],
                    [(obj.attrib['rot'])]
                ]
                curveList.append(curve_data)

  # Построение плана на GUI
  fig = Figure(figsize=(3,4), dpi = 120)
  fig.subplots_adjust(left=0, right=1, bottom=0, top
=1)
  ax = fig.add_subplot(111)

  ax.set_xlim((lineList[-1][0][0] + lineList
[0][0][0])/2 - (3*(lineList[-1][0][1] - lineList
[0][0][1] + (lineList[-1][0][1] - lineList[0][0][1])/10)
)/8, (lineList[-1][0][0] + lineList[0][0][0])/2 + (3*(
lineList[-1][0][1] - lineList[0][0][1] + (lineList[-1][0
 - lineList[0][0][1])/10))/8)
  ax.set_ylim(lineList[0][0][1] - (lineList[-1][0][1]
 - lineList[0][0][1])/20, lineList[-1][0][1] + (
lineList[-1][0][1] - lineList[0][0][1])/20)


  for line in lineList:
      ax.plot([line[0][0], line[1][0]], [line[0][1],
line[1][1]], 'b-')
```

```python

    for spiral in spiralList:
        start = np.array([spiral[0][0], spiral[0][1]])
        end = np.array([spiral[2][0], spiral[2][1]])
        PI = np.array([spiral[1][0], spiral[1][1]])

        def angle_between_points(p1, p2):
            return np.arctan2(p2[1] - p1[1], p2[0] - p1[0])
        theta0 = angle_between_points(start, PI)
        theta1 = angle_between_points(PI, end)

        if spiral[5] == "INF":
            kappa0 = 0
        else:
            kappa0 = 1.0 / float(spiral[5])

        if spiral[4] == "INF":
            kappa1 = 0
        else:
            kappa1 = 1.0 / float(spiral[4])

        if spiral[3] == "cw":
            kappa1 = -abs(kappa1)

        clothoids = SolveG2(
            start[0],
            start[1],
            theta0,
            kappa0,
            end[0],
            end[1],
            theta1,
            kappa1
        )
```

```
        xs, ys = [], []
        for c in clothoids:
            x_sample, y_sample = c.SampleXY(200)
            xs.extend(x_sample)
            ys.extend(y_sample)

        ax.plot(xs, ys, 'r-')



    for curve in curveList:
        radius = (((curve[2][0]) - (curve[0][0]))**2 + ((curve[2][1]) - (curve[0][1]))**2)**0.5
        start = np.array([curve[0][0], curve[0][1]])
        end = np.array([curve[1][0], curve[1][1]])
        vec = end - start
        chord_length = np.linalg.norm(vec)

        midpoint = (start + end) / 2
        perp_vec = np.array([-vec[1], vec[0]])
        perp_vec = perp_vec / np.linalg.norm(perp_vec)

        h = np.sqrt(radius**2 - (chord_length / 2)**2)

        if curve[3][0] == 'ccw':
            center = midpoint - h * perp_vec
        else:
            center = midpoint + h * perp_vec

        def angle(point):
            return np.degrees(np.arctan2(point[1] - center[1], point[0] - center[0]))

        theta1 = angle(start)
        theta2 = angle(end)
```

```
        ax.set_aspect('equal')
        if curve[3][0] == 'ccw':
            arc = Arc(center, 2*radius, 2*radius, angle=0, theta1=theta2, theta2=theta1, color='green', lw=2)
        else:
            arc = Arc(center, 2*radius, 2*radius, angle=0, theta1=theta1, theta2=theta2, color='green', lw=2)
        ax.add_patch(arc)

    scatter_pos = ax.scatter(lineList[0][0][0], lineList[0][0][1], c='green')

    fig.gca().set_facecolor('black')
    ax.spines['top'].set_visible(False)
    ax.spines['bottom'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.spines['left'].set_visible(False)
    ax.tick_params(axis='both', which='both', labelbottom=False, labelleft=False, bottom=False, left=False)
    ax.grid(True)

    plan_plt = FigureCanvasTkAgg(fig, master = window)
    plan_plt.draw()
    plan_plt.get_tk_widget().place(x=49.0, y=168.0)
except:
    print("No LANDXML-file")


    # Текущая позиция 0

# Возвышение рельса
```

```
image_image_100 = PhotoImage(file=relative_to_assets("blind_big.png"))
# image_110
image_100 = canvas.create_image(1009, 330.0, image=image_image_100)

up_right = canvas.create_text((989+1064)/2, (294+324)/2, anchor="center", text="", fill="#000000", font=("Almarai Bold", 15 * -1))

image_image_110 = PhotoImage(file=relative_to_assets("blind_big.png"))
# image_111
image_110 = canvas.create_image(615.5, 330.0, image=image_image_110)

up_left = canvas.create_text((589+665)/2, (294+324)/2, anchor="center", text="", fill="#000000", font=("Almarai Bold", 16 * -1))


# Изображения для GUI
image_image_1 = PhotoImage(file=relative_to_assets("image_1.png"))
image_1 = canvas.create_image(812.0, 395.0, image=image_image_1)

image_image_2 = PhotoImage(file=relative_to_assets("image_2.png"))
image_2 = canvas.create_image(813.0, 502.0, image=image_image_2)

image_image_3 = PhotoImage(file=relative_to_assets("image_3.png"))
image_3 = canvas.create_image(808.0, 569.0, image=image_image_3)
```

```
image_image_4 = PhotoImage(file=relative_to_assets("
    image_4.png"))
image_4 = canvas.create_image(773.0, 322.0, image=
    image_image_4)

# Координаты оси пути
canvas.create_rectangle(583.0, 584.0, 733.0, 614.0,
    fill="#D9D9D9", outline="black", width=2)
text_x_center = canvas.create_text((583+733)/2,
    (584+614)/2 ,anchor="center",text="0",fill="#000000"
    ,font=("Almarai Bold", 18 * -1))

canvas.create_rectangle(733.0, 584.0, 883.0, 614.0,fill
    ="#D9D9D9", outline="black", width=2)
text_y_center = canvas.create_text((733+883)/2,
    (584+614)/2, anchor="center", text="0", fill="
    #000000", font=("Almarai Bold", 18 * -1))

canvas.create_rectangle(883.0, 584.0, 1033.0, 614.0,
    fill="#D9D9D9", outline="black", width=2)
text_z_center = canvas.create_text((883+1033)/2,
    (584+614)/2, anchor="center", text="0", fill="
    #000000", font=("Almarai Bold", 18 * -1))

# Ширина колеи
canvas.create_rectangle(848.0, 307.0, 923.0, 337.0,
    fill="#D9D9D9", outline="black", width=2)
text_rail_width = canvas.create_text((848+923)/2,
    (307+337)/2, anchor="center", text="0", fill="
    #000000", font=("Almarai Bold", 15 * -1))

# Кнопка Старт""
button_image_start = PhotoImage(file=relative_to_assets
    ("button_1.png"))
```

```python
button_start = Button(image=button_image_start, borderwidth=0, activebackground="#004689", highlightthickness=0, command=change_start, relief="flat")
button_start.place(x=1120.0, y=668.0, width=125.0, height=35.0)

# Кнопка Стоп""
button_image_stop = PhotoImage(file=relative_to_assets("button_3.png"))
button_stop = Button(image=button_image_stop, borderwidth=0, activebackground="#004689", highlightthickness=0, command=change_stop, relief="flat")
button_stop.place(x=47.0, y=669.0, width=125.0, height=35.0)

image_image_5 = PhotoImage(file=relative_to_assets("image_5.png"))
image_5 = canvas.create_image(229.0, 408.0, image=image_image_5)

image_image_6 = PhotoImage(file=relative_to_assets("image_6.png"))
image_6 = canvas.create_image(812.0, 352.0, image=image_image_6)

# Подъемка оси пути
image_image_7 = PhotoImage(file=relative_to_assets("image_7.png"))
image_7 = canvas.create_image(633, 517, image=image_image_7)

# Сдвижка оси пути
```

```
image_image_8 = PhotoImage(file=relative_to_assets("image_8.png"))
image_image_87 = PhotoImage(file=relative_to_assets("image_87.png")) # Влево
image_image_88 = PhotoImage(file=relative_to_assets("image_88.png")) # Вправо
image_8 = canvas.create_image(995.0, 517.0, image=image_image_8)

# Сдвижка
image_image_9 = PhotoImage(file=relative_to_assets("image_9.png"))
image_9 = canvas.create_image(748.0, 484.0, image=image_image_9)

canvas.create_rectangle(647, 502, 722, 532, fill="#D9D9D9", outline="black", width=2)
text_right_rail_up = canvas.create_text((647+722)/2, (502+532)/2, anchor="center", text="0", fill="#000000", font=("Almarai Bold", 15 * -1))

# Подъемка
image_image_10 = PhotoImage(file=relative_to_assets("image_10.png"))
image_10 = canvas.create_image(878.0, 484.0, image=image_image_10)

canvas.create_rectangle(906, 502, 981, 532, fill="#D9D9D9", outline="black", width=2)
text_right_rail_side = canvas.create_text((906+981)/2, (502+532)/2, anchor="center", text="0", fill="#000000", font=("Almarai Bold", 15 * -1))

image_image_11 = PhotoImage(file=relative_to_assets("image_11.png"))
```

```
image_11 = canvas.create_image(640.0, 68.0, image=image_image_11)

image_image_12 = PhotoImage(file=relative_to_assets("image_12.png"))
image_12 = canvas.create_image(662.0, 216.0, image=image_image_12)

image_image_13 = PhotoImage(file=relative_to_assets("image_13.png"))
image_13 = canvas.create_image(587.0, 246.0, image=image_image_13)

# Тип следующего участка пути
image_image_14 = PhotoImage(file=relative_to_assets("image_14.png"))
image_14 = canvas.create_image(737.0, 246.0, image=image_image_14)

text_next_status = canvas.create_text((662+812)/2, (231+261)/2, anchor="center", text=" ", fill="#000000", font=("Almarai Bold", 22 * -1))

# Расстояние до следующего участка пути
image_image_15 = PhotoImage(file=relative_to_assets("image_15.png"))
image_15 = canvas.create_image(962.0, 246.0, image=image_image_15)

text_next_way = canvas.create_text((812+1112)/2, (231+261)/2, anchor="center", text="метров", fill="#000000", font=("Almarai Bold", 22 * -1))

# Текущий участок пути
```

```
image_image_16 = PhotoImage(file=relative_to_assets("image_16.png"))
image_16 = canvas.create_image(962.0, 216.0, image=image_image_16)

text_way_status = canvas.create_text((812+1112)/2, (201+231)/2, anchor="center", text=" ", fill="#000000", font=("Almarai Bold", 22 * -1))

# Точка положения на плане
plan_pos = canvas.create_oval(222.0, 401.0, 232.0, 411.0, fill="lightcoral", outline="red")

# Текущее время
image_image_24 = PhotoImage(file=relative_to_assets("image_24.png"))
image_24 = canvas.create_image(1031.0, 156.0, image=image_image_24)

canvas.create_rectangle(1105.0, 141.0, 1255.0, 171.0, fill="#D9D9D9", outline="")
text_time = canvas.create_text((1105+1255)/2, (141+171)/2, anchor="center", text="00:00:00", fill="#000000", font=("Almarai Bold", 22 * -1))

# Консоль
consol_box = Text(window, wrap='word', height=50, width=430, bg='black', fg='white', font=('Consolas', 10))
consol_box.place(x=(733+883)/2, y=(584+614)/2 + 50, anchor="center", width=450.0, height= 60.0)

sys.stdout = ConsoleRedirector(consol_box)
sys.stderr = ConsoleRedirector(consol_box)
```

```
image_bind_110 = PhotoImage(file=relative_to_assets("image_111.png"))
image_bind_100 = PhotoImage(file=relative_to_assets("image_110.png"))
image_blind = PhotoImage(file=relative_to_assets("blind_big.png"))

#обновление------------------ значений окон---------------------------------
def update_pos():
    #Time
    current_time = time.strftime("%H:%M:%S")
    canvas.itemconfig(text_time, text=f"{current_time}")

    global counter, flag_connect_lost, plan_plt, ax, scatter_pos

    if flag_start_visual == 1 and flag_connect_lost == 1 and flag_retry == 0:
        change_start()

    if flag_start_visual == 1 and flag_connect_ok == 1:

        x_center = round(C_R[0], 4)
        y_center = round(C_R[1], 4)
        z_center = round(C_R[2], 4)

        right_rail_side = round(center_shift,1)

        right_rail_up = round(height_shift,1)

        rail_width = round((path_W*10**(3)),1)

        # Смена возвышения
```

```
        try:
            if rail_height < 0:
                canvas.itemconfig(up_left , text=f"{round(-rail_height*1000,2)} мм")
                canvas.itemconfig(up_right , text=f"")
                canvas.itemconfig(image_110, image=image_bind_110)
                canvas.itemconfig(image_100, image=image_blind)
            else:
                canvas.itemconfig(up_left , text=f"")
                canvas.itemconfig(up_right , text=f"{round(rail_height*1000,2)} мм")
                canvas.itemconfig(image_110, image=image_blind)
                canvas.itemconfig(image_100, image=image_bind_100)
        except:
            print("Ошибка при смене возвышения")

        if landxml_status == "on_track":

            next_way = round(((next_coor[0] - C_R[0])**2 + (next_coor[1] - C_R[1])**2)**0.5, 3)

            if next_status == 'line':
                next_t_status = "прямой"
            elif next_status == 'curve':
                next_t_status = "кривой"
            else:
                next_t_status = "ПК"

            if way_status == 'line':
                cur_w = "прямая"
            elif way_status == 'curve':
```

```
                cur_w = "кривая"
            else:
                cur_w = "переходная кривая"
        else:
            next_way = "No LANDXML"
            next_t_status = "No LANDXML"
            cur_w = "No LANDXML"

        # Сдвижка и подъемка
        try:
            if center_shift < 0:
                canvas.itemconfig(image_8, image=
image_image_87)
                canvas.itemconfig(text_right_rail_side,
 text=f"{-right_rail_side} м")
            else:
                canvas.itemconfig(image_8, image=
image_image_88)
                canvas.itemconfig(text_right_rail_side,
 text=f"{right_rail_side} м")
        except:
            print("Ошибка при смене сдвижки")


        canvas.itemconfig(text_right_rail_up, text=f"{
right_rail_up} м")

        # Ширина колеи
        canvas.itemconfig(text_rail_width, text=f"{
rail_width} мм")

        # Тип следующего участок пути
        canvas.itemconfig(text_next_status, text=f"{
next_t_status}")
```

```
        # Расстояние до следующего участка пути
        canvas.itemconfig(text_next_way, text=f"{next_way} метров")

        # Текущий тип участка пути
        canvas.itemconfig(text_way_status, text=f"{cur_w}")

        # Координаты оси пути
        canvas.itemconfig(text_x_center, text=f"{x_center} м")
        canvas.itemconfig(text_y_center, text=f"{y_center} м")
        canvas.itemconfig(text_z_center, text=f"{z_center} м")

        # Обновление плана
        try:
            try:
                scatter_pos.remove()
            finally:
                ax.set_xlim(C_R[0] - 0.75*25, C_R[0] + 0.75*25)
                ax.set_ylim(C_R[1] - 25, C_R[1] + 25)
                scatter_pos = ax.scatter(C_R[0], C_R[1], c='green')
                plan_plt.draw()
        except:
            pass

        if flag_prism_lock == 0:
            canvas.itemconfig(text_x_center, text=f"")
            canvas.itemconfig(text_y_center, text=f"Цель потеряна")
            canvas.itemconfig(text_z_center, text=f"")
```

```
    window.after(100, update_pos)

def app_end():
    global flag_app_end
    flag_app_end = 1
    server_socket.close()
    thread1.join()
    thread2.join()
    thread3.join()
    window.destroy()

update_pos()
window.resizable(False, False)
window.protocol("WM_DELETE_WINDOW", app_end)
window.mainloop()

if flag_app_end == 1:
    app_end()
```

Файл HMI-Tah/src/**pygeocom.py**

```
from typing import Tuple, Any
from enum import Enum, IntFlag
from datetime import datetime
from collections import namedtuple
from collections.abc import Callable

GRC_TPS = 0x0000   # main return codes (identical to
  RC_SUP!!)
GRC_SUP = 0x0000   # supervisor task (identical to
  RCBETA!!)
GRC_ANG = 0x0100   # angle- and inclination
GRC_ATA = 0x0200   # automatic target acquisition
GRC_EDM = 0x0300   # electronic distance meter
GRC_GMF = 0x0400   # geodesy mathematics & formulas
GRC_TMC = 0x0500   # measurement & calc
GRC_MEM = 0x0600   # memory management
GRC_MOT = 0x0700   # motorization
GRC_LDR = 0x0800   # program loader
GRC_BMM = 0x0900   # basics of man machine interface
GRC_TXT = 0x0A00   # text management
GRC_MMI = 0x0B00   # man machine interface
GRC_COM = 0x0C00   # communication
GRC_DBM = 0x0D00   # data base management
GRC_DEL = 0x0E00   # dynamic event logging
GRC_FIL = 0x0F00   # file system
GRC_CSV = 0x1000   # central services
GRC_CTL = 0x1100   # controlling task
GRC_STP = 0x1200   # start + stop task
GRC_DPL = 0x1300   # data pool
GRC_WIR = 0x1400   # wi registration
GRC_USR = 0x2000   # user task
GRC_ALT = 0x2100   # alternate user task
GRC_AUT = 0x2200   # automatization
```

```
GRC_AUS = 0x2300   # alternative user
GRC_BAP = 0x2400   # basic applications
GRC_SAP = 0x2500   # system applications
GRC_COD = 0x2600   # standard code function
GRC_BAS = 0x2700   # GeoBasic interpreter
GRC_IOS = 0x2800   # Input-/ output- system
GRC_CNF = 0x2900   # configuration facilities
GRC_XIT = 0x2E00   # XIT subsystem (Excite-Level LIS)
GRC_DNA = 0x2F00   # DNA2 subsystem
GRC_ICD = 0x3000   # cal data management
GRC_KDM = 0x3100   # keyboard display module
GRC_LOD = 0x3200   # firmware loader
GRC_FTR = 0x3300   # file transfer
GRC_VNF = 0x3F00   # reserved for new TPS1200 subsystem
GRC_GPS = 0x4000   # GPS subsystem
GRC_TST = 0x4100   # Test subsystem
GRC_PTF = 0x4F00   # reserved for new GPS1200 subsystem
GRC_APP = 0x5000   # offset for all applications
GRC_RES = 0x7000   # reserved code range

class ReturnCode(Enum):
    GRC_OK                  = GRC_TPS +  0   # Function
  successfully completed.
    GRC_UNDEFINED           = GRC_TPS +  1   # Unknown
  error result unspecified.
    GRC_IVPARAM             = GRC_TPS +  2   # Invalid
  parameter detected.\nResult unspecified.
    GRC_IVRESULT            = GRC_TPS +  3   # Invalid
  result.
    GRC_FATAL               = GRC_TPS +  4   # Fatal error.
    GRC_NOT_IMPL            = GRC_TPS +  5   # Not
  implemented yet.
    GRC_TIME_OUT            = GRC_TPS +  6   # Function
  execution timed out.\nResult unspecified.
```

```
  GRC_SET_INCOMPL      = GRC_TPS +  7   # Parameter setup for subsystem is incomplete.
  GRC_ABORT            = GRC_TPS +  8   # Function execution has been aborted.
  GRC_NOMEMORY         = GRC_TPS +  9   # Fatal error - not enough memory.
  GRC_NOTINIT          = GRC_TPS + 10   # Fatal error - subsystem not initialized.
  GRC_SHUT_DOWN        = GRC_TPS + 12   # Subsystem is down.
  GRC_SYSBUSY          = GRC_TPS + 13   # System busy/already in use of another process.\nCannot execute function.
  GRC_HWFAILURE        = GRC_TPS + 14   # Fatal error - hardware failure.
  GRC_ABORT_APPL       = GRC_TPS + 15   # Execution of application has been aborted (SHIFT-ESC).
  GRC_LOW_POWER        = GRC_TPS + 16   # Operation aborted - insufficient power supply level.
  GRC_IVVERSION        = GRC_TPS + 17   # Invalid version of file ...
  GRC_BATT_EMPTY       = GRC_TPS + 18   # Battery empty
  GRC_NO_EVENT         = GRC_TPS + 20   # no event pending.
  GRC_OUT_OF_TEMP      = GRC_TPS + 21   # out of temperature range
  GRC_INSTRUMENT_TILT  = GRC_TPS + 22   # intrument tilting out of range
  GRC_COM_SETTING      = GRC_TPS + 23   # communication error
  GRC_NO_ACTION        = GRC_TPS + 24   # GRC_TYPE Input 'do no action'
  GRC_SLEEP_MODE       = GRC_TPS + 25   # Instr. run into the sleep mode
```

```
  GRC_NOTOK              = GRC_TPS + 26   # Function not successfully completed.
  GRC_NA                 = GRC_TPS + 27   # Not available
  GRC_OVERFLOW           = GRC_TPS + 28   # Overflow error
  GRC_STOPPED            = GRC_TPS + 29   # System or subsystem has been stopped

  GRC_COM_ERO                        = GRC_COM +  0  # Initiate Extended Runtime Operation (ERO).
  GRC_COM_CANT_ENCODE                = GRC_COM +  1  # Cannot encode arguments in client.
  GRC_COM_CANT_DECODE                = GRC_COM +  2  # Cannot decode results in client.
  GRC_COM_CANT_SEND                  = GRC_COM +  3  # Hardware error while sending.
  GRC_COM_CANT_RECV                  = GRC_COM +  4  # Hardware error while receiving.
  GRC_COM_TIMEDOUT                   = GRC_COM +  5  # Request timed out.
  GRC_COM_WRONG_FORMAT               = GRC_COM +  6  # Packet format error.
  GRC_COM_VER_MISMATCH               = GRC_COM +  7  # Version mismatch between client and server.
  GRC_COM_CANT_DECODE_REQ            = GRC_COM +  8  # Cannot decode arguments in server.
  GRC_COM_PROC_UNAVAIL               = GRC_COM +  9  # Unknown RPC, procedure ID invalid.
  GRC_COM_CANT_ENCODE_REP            = GRC_COM + 10  # Cannot encode results in server.
  GRC_COM_SYSTEM_ERR                 = GRC_COM + 11  # Unspecified generic system error.
  GRC_COM_UNKNOWN_HOST               = GRC_COM + 12  #  (Unused error code)
```

```
  GRC_COM_FAILED                   = GRC_COM + 13  # Unspecified error.
  GRC_COM_NO_BINARY                = GRC_COM + 14  # Binary protocol not available.
  GRC_COM_INTR                     = GRC_COM + 15  # Call interrupted.
  GRC_COM_UNKNOWN_ADDR             = GRC_COM + 16  # (Unused error code)
  GRC_COM_NO_BROADCAST             = GRC_COM + 17  # (Unused error code)
  GRC_COM_REQUIRES_8DBITS          = GRC_COM + 18  # Protocol needs 8bit encoded chararacters.
  GRC_COM_UD_ERROR                 = GRC_COM + 19  # (Unused error code)
  GRC_COM_LOST_REQ                 = GRC_COM + 20  # (Unused error code)
  GRC_COM_TR_ID_MISMATCH           = GRC_COM + 21  # Transacation ID mismatch error.
  GRC_COM_NOT_GEOCOM               = GRC_COM + 22  # Protocol not recognizeable.
  GRC_COM_UNKNOWN_PORT             = GRC_COM + 23  # (WIN) Invalid port address.
  GRC_COM_ILLEGAL_TRPT_SELECTOR    = GRC_COM + 24  # (Unused error code)
  GRC_COM_TRPT_SELECTOR_IN_USE     = GRC_COM + 25  # (Unused error code)
  GRC_COM_INACTIVE_TRPT_SELECTOR = GRC_COM + 26 #  (Unused error code)
  GRC_COM_ERO_END                  = GRC_COM + 27  # ERO is terminating.
  GRC_COM_OVERRUN                  = GRC_COM + 28  # Internal error: data buffer overflow.
  GRC_COM_SRVR_RX_CHECKSUM_ERROR = GRC_COM + 29  # Invalid checksum on server side received.
```

```
  GRC_COM_CLNT_RX_CHECKSUM_ERROR = GRC_COM + 30  #
Invalid checksum on client side received.
  GRC_COM_PORT_NOT_AVAILABLE     = GRC_COM + 31  #  (
WIN) Port not available.
  GRC_COM_PORT_NOT_OPEN          = GRC_COM + 32  #  (
WIN) Port not opened.
  GRC_COM_NO_PARTNER             = GRC_COM + 33  #  (
WIN) Unable to find TPS.
  GRC_COM_ERO_NOT_STARTED        = GRC_COM + 34  #
Extended Runtime Operation could not be started.
  GRC_COM_CONS_REQ               = GRC_COM + 35  #
Att to send cons reqs
  GRC_COM_SRVR_IS_SLEEPING       = GRC_COM + 36   #
TPS has gone to sleep. Wait and try again.
  GRC_COM_SRVR_IS_OFF            = GRC_COM + 37  #
TPS has shut down. Wait and try again.

  GRC_EDM_SYSTEM_ERR         = GRC_EDM + 1 # Fatal
EDM sensor error. See for the exact reason the
original EDM sensor error number. In the most cases a
 service problem.
  # Sensor user errors
  GRC_EDM_INVALID_COMMAND    = GRC_EDM + 2 # Invalid
command or unknown command, see command syntax.
  GRC_EDM_BOOM_ERR           = GRC_EDM + 3 #
Boomerang error.
  GRC_EDM_SIGN_LOW_ERR       = GRC_EDM + 4 # Received
 signal to low, prisma to far away, or natural
barrier, bad environment, etc.
  GRC_EDM_DIL_ERR            = GRC_EDM + 5 # Obsolete
  GRC_EDM_SIGN_HIGH_ERR      = GRC_EDM + 6 # Received
 signal to strong, prism too near, stranger light
effect.
  # New TPS1200 sensor user errors
```

```
  GRC_EDM_TIMEOUT              = GRC_EDM + 7 # Timeout, measuring time exceeded (signal too weak, beam interrupted,..)
  GRC_EDM_FLUKT_ERR            = GRC_EDM + 8 # Too much turbulences or distractions
  GRC_EDM_FMOT_ERR             = GRC_EDM + 9 # Filter motor defective

  # Subsystem errors
  GRC_EDM_DEV_NOT_INSTALLED    = GRC_EDM + 10 # Device like EGL, DL is not installed.
  GRC_EDM_NOT_FOUND            = GRC_EDM + 11 # Search result invalid. For the exact explanation \nsee in the description of the called function.
  GRC_EDM_ERROR_RECEIVED       = GRC_EDM + 12 # Communication ok, but an error\nreported from the EDM sensor.
  GRC_EDM_MISSING_SRVPWD       = GRC_EDM + 13 # No service password is set.
  GRC_EDM_INVALID_ANSWER       = GRC_EDM + 14 # Communication ok, but an unexpected\nanswer received.
  GRC_EDM_SEND_ERR             = GRC_EDM + 15 # Data send error, sending buffer is full.
  GRC_EDM_RECEIVE_ERR          = GRC_EDM + 16 # Data receive error, like\nparity buffer overflow.
  GRC_EDM_INTERNAL_ERR         = GRC_EDM + 17 # Internal EDM subsystem error.
  GRC_EDM_BUSY                 = GRC_EDM + 18 # Sensor is working already,\nabort current measuring first.
  GRC_EDM_NO_MEASACTIVITY      = GRC_EDM + 19 # No measurement activity started.
  GRC_EDM_CHKSUM_ERR           = GRC_EDM + 20 # Calculated checksum, resp. received data wrong\n(only in binary communication mode possible).
```

```
  GRC_EDM_INIT_OR_STOP_ERR   = GRC_EDM + 21 # During start up or shut down phase an\nerror occured. It is saved in the DEL buffer.
  GRC_EDM_SRL_NOT_AVAILABLE  = GRC_EDM + 22 # Red laser not available on this sensor HW.
  GRC_EDM_MEAS_ABORTED       = GRC_EDM + 23 # Measurement will be aborted (will be used for the lasersecurity)

  # New TPS1200 sensor user error
  GRC_EDM_SLDR_TRANSFER_PENDING = GRC_EDM + 30 # Multiple OpenTransfer calls.
  GRC_EDM_SLDR_TRANSFER_ILLEGAL = GRC_EDM + 31 # No opentransfer happened.
  GRC_EDM_SLDR_DATA_ERROR       = GRC_EDM + 32 # Unexpected data format received.
  GRC_EDM_SLDR_CHK_SUM_ERROR    = GRC_EDM + 33 # Checksum error in transmitted data.
  GRC_EDM_SLDR_ADDR_ERROR       = GRC_EDM + 34 # Address out of valid range.
  GRC_EDM_SLDR_INV_LOADFILE     = GRC_EDM + 35 # Firmware file has invalid format.
  GRC_EDM_SLDR_UNSUPPORTED      = GRC_EDM + 36 # Current (loaded) firmware doesn't support upload.
  GRC_EDM_UNKNOW_ERR            = GRC_EDM + 40 # Undocumented error from the\nEDM sensor, should not occur.

  GRC_EDM_DISTRANGE_ERR         = GRC_EDM + 50 # Out of distance range (dist too small or large)
  GRC_EDM_SIGNTONOISE_ERR       = GRC_EDM + 51 # Signal to noise ratio too small
  GRC_EDM_NOISEHIGH_ERR         = GRC_EDM + 52 # Noise to high
```

```
  GRC_EDM_PWD_NOTSET             = GRC_EDM + 53 # Password is not set
  GRC_EDM_ACTION_NO_MORE_VALID   = GRC_EDM + 54 # Elapsed time between prepare und start fast measurement for ATR to long
  GRC_EDM_MULTRG_ERR             = GRC_EDM + 55 # Possibly more than one target (also a sensor error)

  GRC_MOT_UNREADY     = GRC_MOT + 0 # motorization is not ready (1792)
  GRC_MOT_BUSY        = GRC_MOT + 1 # motorization is handling another task (1793)
  GRC_MOT_NOT_OCONST  = GRC_MOT + 2 # motorization is not in velocity mode (1794)
  GRC_MOT_NOT_CONFIG  = GRC_MOT + 3 # motorization is in the wrong mode or busy (1795)
  GRC_MOT_NOT_POSIT   = GRC_MOT + 4 # motorization is not in posit mode (1796)
  GRC_MOT_NOT_SERVICE = GRC_MOT + 5 # motorization is not in service mode (1797)
  GRC_MOT_NOT_BUSY    = GRC_MOT + 6 # motorization is handling no task (1798)
  GRC_MOT_NOT_LOCK    = GRC_MOT + 7 # motorization is not in tracking mode (1799)
  GRC_MOT_NOT_SPIRAL  = GRC_MOT + 8 # motorization is not in spiral mode (1800)

  GRC_TMC_NO_FULL_CORRECTION = GRC_TMC + 3      # Warning: measurment without full correction
  GRC_TMC_ACCURACY_GUARANTEE = GRC_TMC + 4      # Info   : accuracy can not be guarantee

  GRC_TMC_ANGLE_OK               = GRC_TMC + 5   # Warning: only angle measurement valid
```

```
  GRC_TMC_ANGLE_NOT_FULL_CORR   = GRC_TMC + 8   # Warning: only angle measurement valid but without full correction
  GRC_TMC_ANGLE_NO_ACC_GUARANTY = GRC_TMC + 9   # Info   : only angle measurement valid but accuracy can not be guarantee

  GRC_TMC_ANGLE_ERROR       = GRC_TMC + 10      # Error  : no angle measurement

  GRC_TMC_DIST_PPM          = GRC_TMC + 11      # Error  : wrong setting of PPM or MM on EDM
  GRC_TMC_DIST_ERROR        = GRC_TMC + 12      # Error  : distance measurement not done (no aim, etc.)
  GRC_TMC_BUSY              = GRC_TMC + 13      # Error  : system is busy (no measurement done)
  GRC_TMC_SIGNAL_ERROR      = GRC_TMC + 14      # Error  : no signal on EDM (only in signal mode)

  GRC_BMM_XFER_PENDING               = GRC_BMM +  1 # Loading process already opened
  GRC_BMM_NO_XFER_OPEN               = GRC_BMM +  2 # Transfer not opened
  GRC_BMM_UNKNOWN_CHARSET            = GRC_BMM +  3 # Unknown character set
  GRC_BMM_NOT_INSTALLED              = GRC_BMM +  4 # Display module not present
  GRC_BMM_ALREADY_EXIST              = GRC_BMM +  5 # Character set already exists
  GRC_BMM_CANT_DELETE                = GRC_BMM +  6 # Character set cannot be deleted
  GRC_BMM_MEM_ERROR                  = GRC_BMM +  7 # Memory cannot be allocated
```

```
  GRC_BMM_CHARSET_USED             = GRC_BMM +  8 #
Character set still used
  GRC_BMM_CHARSET_SAVED            = GRC_BMM +  9 #
Charset cannot be deleted or is protected
  GRC_BMM_INVALID_ADR              = GRC_BMM + 10 #
Attempt to copy a character block\noutside the
allocated memory
  GRC_BMM_CANCELANDADR_ERROR       = GRC_BMM + 11 #
Error during release of allocated memory
  GRC_BMM_INVALID_SIZE             = GRC_BMM + 12 #
Number of bytes specified in header\ndoes not match
the bytes read
  GRC_BMM_CANCELANDINVSIZE_ERROR = GRC_BMM + 13 #
Allocated memory could not be released
  GRC_BMM_ALL_GROUP_OCC            = GRC_BMM + 14 # Max
. number of character sets already loaded
  GRC_BMM_CANT_DEL_LAYERS          = GRC_BMM + 15 #
Layer cannot be deleted
  GRC_BMM_UNKNOWN_LAYER            = GRC_BMM + 16 #
Required layer does not exist
  GRC_BMM_INVALID_LAYERLEN         = GRC_BMM + 17 #
Layer length exceeds maximum

  AUT_RC_TIMEOUT          = GRC_AUT +  4 # Timeout,
no target found
  AUT_RC_DETENT_ERROR     = GRC_AUT +  5 #
  AUT_RC_ANGLE_ERROR      = GRC_AUT +  6 #
  AUT_RC_MOTOR_ERROR      = GRC_AUT +  7 #
Motorisation error
  AUT_RC_INCACC           = GRC_AUT +  8 #
  AUT_RC_DEV_ERROR        = GRC_AUT +  9 # Deviation
measurement error
  AUT_RC_NO_TARGET        = GRC_AUT + 10 # No target
detected
```

```
    AUT_RC_MULTIPLE_TARGETS = GRC_AUT + 11 # Multiple targets detected
    AUT_RC_BAD_ENVIRONMENT  = GRC_AUT + 12 # Bad environment conditions
    AUT_RC_DETECTOR_ERROR   = GRC_AUT + 13 #
    AUT_RC_NOT_ENABLED      = GRC_AUT + 14 #
    AUT_RC_CALACC           = GRC_AUT + 15 #
    AUT_RC_ACCURACY         = GRC_AUT + 16 # Position not exactly reached

class byte(int):
    def __new__(cls, value, *args, **kwargs):
        if (type(value) == str):
            value = int(value.strip("'"), 16)
        elif (type(value) == bytes):
            value = int(value.strip(b"'"), 16)

        if value < 0:
            raise ValueError("byte types must not be less than zero")
        if value > 255:
            raise ValueError("byte types must not be more than 255 (0xff)")

        return  super(cls, cls).__new__(cls, value)

class PrismType(Enum):
    LEICA_ROUND       =  0 # Prism type: Leica circular prism
    LEICA_MINI        =  1 # Prism type: Leica mini prism
    LEICA_TAPE        =  2 # Prism type: Leica reflective tape
    LEICA_360         =  3 # Prism type: Leica 360° prism
```

```
    USER1              =  4 # Prism type: User defined 1
    USER2              =  5 # Prism type: User defined 2
    USER3              =  6 # Prism type: User defined 3
    LEICA_360_MINI     =  7 # Prism type: Leica 360° mini
    LEICA_MINI_ZERO    =  8 # Prism type: Leica mini zero
    LEICA_USER         =  9 # Prism type: user???
    LEICA_HDS_TAPE     = 10 # Prism type: tape cyra???
    LEICA_GRZ121_ROUND = 11 # Prism type: Leica GRZ121 round for machine guidance

class ReflectorType(Enum):
    UNDEFINED = 0
    PRISM = 1
    TAPE = 2

class TargetType(Enum):
    REFLECTOR = 0
    REFLECTORLESS = 1

class InclinationSensorProgram(Enum):
    TMC_MEA_INC = 0     # Use sensor (apriori sigma)
    TMC_AUTO_INC = 1    # Automatic mode (sensor/plane)
    TMC_PLANE_INC = 2   # Use plane (apriori sigma)

class EDMMode(Enum):
    EDM_MODE_NOT_USED = 0   # Init value
    EDM_SINGLE_TAPE = 1     # Single measurement with tape
    EDM_SINGLE_STANDARD = 2 # Standard single measurement
```

```
    EDM_SINGLE_FAST = 3      # Fast single measurement
    EDM_SINGLE_LRANGE = 4    # Long range single measurement
    EDM_SINGLE_SRANGE = 5    # Short range single measurement
    EDM_CONT_STANDARD = 6    # Standard repeated measurement
    EDM_CONT_DYNAMIC = 7     # Dynamic repeated measurement
    EDM_CONT_REFLESS = 8     # Reflectorless repeated measurement
    EDM_CONT_FAST = 9        # Fast repeated measurement
    EDM_AVERAGE_IR = 10      # Standard average measurement
    EDM_AVERAGE_SR = 11      # Short range average measurement
    EDM_AVERAGE_LR = 12      # Long range average measurement

class DeviceClass(Enum):
    # TPS1000 Family         ------------------------ accuracy
    TPS_CLASS_1100 = 0       # TPS1000 family member, 1 mgon,   3"
    TPS_CLASS_1700 = 1       # TPS1000 family member, 0.5 mgon, 1.5"
    TPS_CLASS_1800 = 2       # TPS1000 family member, 0.3 mgon, 1"
    TPS_CLASS_5000 = 3       # TPS2000 family member
    TPS_CLASS_6000 = 4       # TPS2000 family member
    TPS_CLASS_1500 = 5       # TPS1000 family member
    TPS_CLASS_2003 = 6       # TPS2000 family member
    TPS_CLASS_5005 = 7       # TPS5000      "
    TPS_CLASS_5100 = 8       # TPS5000      "
```

```
    # TPS1100 Family       ------------------------
  accuracy
    TPS_CLASS_1102 = 100    # TPS1000 family member, 2"
    TPS_CLASS_1103 = 101    # TPS1000 family member, 3"
    TPS_CLASS_1105 = 102    # TPS1000 family member, 5"
    TPS_CLASS_1101 = 103    # TPS1000 family member,
  1."

    # TPS1200 Family       ------------------------
  accuracy
    TPS_CLASS_1202 = 200    # TPS1200 family member, 2"
    TPS_CLASS_1203 = 201    # TPS1200 family member, 3"
    TPS_CLASS_1205 = 202    # TPS1200 family member, 5"
    TPS_CLASS_1201 = 203    # TPS1200 family member, 1"

class DeviceType(IntFlag):
    # TPS1x00 common
    TPS_DEVICE_T   = 0x00000   # Theodolite without
  built-in EDM
    TPS_DEVICE_MOT = 0x00004   # Motorized device
    TPS_DEVICE_ATR = 0x00008   # Automatic Target
  Recognition
    TPS_DEVICE_EGL = 0x00010   # Electronic Guide Light
    TPS_DEVICE_DB  = 0x00020   # reserved (Database,
  not GSI)
    TPS_DEVICE_DL  = 0x00040   # Diode laser
    TPS_DEVICE_LP  = 0x00080   # Laser plumbed

    # TPS1000 specific
    TPS_DEVICE_TC1 = 0x00001   # tachymeter (TCW1)
    TPS_DEVICE_TC2 = 0x00002   # tachymeter (TCW2)

    # TPS1100/TPS1200 specific
    TPS_DEVICE_TC     = 0x00001 # tachymeter (TCW3)
```

```
    TPS_DEVICE_TCR    = 0x00002 # tachymeter (TCW3 with red laser)
    TPS_DEVICE_ATC    = 0x00100 # Autocollimation lamp (used only PMU)
    TPS_DEVICE_LPNT   = 0x00200 # Laserpointer
    TPS_DEVICE_RL_EXT = 0x00400 # Reflectorless EDM with extended range (Pinpoint R100,R300)
    TPS_DEVICE_PS     = 0x00800 # Power Search

    # TPSSim specific
    TPS_DEVICE_SIM = 0x04000     # runs on Simulation, no Hardware

class PowerPath(Enum):
    CURRENT_POWER = 0
    EXTERNAL_POWER = 1
    INTERNAL_POWER = 2

class RecordFormat(Enum):
    GSI_8  = 0
    GSI_16 = 1

class TPSStatus(Enum):
    OFF      = 0
    SLEEPING = 1
    ONLINE   = 2
    LOCAL    = 3
    UNKNOWN  = 4

class OnOff(Enum):
    OFF = 0
    ON = 1

class EGLIntensity(Enum):
    OFF  = 0
```

```
    LOW  = 1
    MID  = 2
    HIGH = 3

class ControllerMode(Enum):
    RELATIVE_POSITIONING = 0
    CONSTANT_SPEED       = 1
    MANUAL_POSITIONING   = 2
    LOCK_IN              = 3
    BRAKE                = 4
    TERMINATE            = 7

class ControllerStopMode(Enum):
    NORMAL   = 0
    SHUTDOWN = 1

class LockInStatus(Enum):
    LOCKED_OUT = 0
    LOCKED_IN  = 1
    PREDICTION = 2

class MeasurementMode(Enum):
    NO_MEASUREMENTS  = 0 # No measurements, take last one
    NO_DISTANCE      = 1 # No distance measurement, angles only
    DEFAULT_DISTANCE = 2 # Default distance measurements, pre-defined using MeasurementProgram
    CLEAR_DISTANCE   = 5 # Clear distances
    STOP_TRACKING    = 6 # Stop tracking laser

class MeasurementProgram(Enum):
    SINGLE_REF_STANDARD  = 0 # standard single IR distance with reflector
```

```
    SINGLE_REF_FAST      = 1 # fast single IR distance with reflector
    SINGLE_REF_VISIBLE   = 2 # long range distance with reflector (red laser)
    SINGLE_RLESS_VISIBLE = 3 # single RL distance reflector free (red laser)
    CONT_REF_STANDARD    = 4 # tracking IR distance with reflector
    CONT_REF_FAST        = 5 # fast tracking IR distance with reflector
    CONT_RLESS_VISIBLE   = 6 # fast tracking RL distance reflector free (red)
    AVG_REF_STANDARD     = 7 # Average IR distance with reflector
    AVG_REF_VISIBLE      = 8 # Average long range dist. with reflector (red)
    AVG_RLESS_VISIBLE    = 9 # Average RL distance reflector free (red laser)

class PositionMode(Enum):
    NORMAL  = 0
    PRECISE = 1

class FineAdjustPositionMode(Enum):
    NORM   = 0 # Angle tolerance
    POINT  = 1 # Point tolerance
    DEFINE = 2 # System independent positioning tolerance; set wit PyGeoCom.set_tolerance()

class ATRRecognitionMode(Enum):
    POSITION = 0 # Positioning to the horizontal and vertical angle
    TARGET   = 1 # Positioning to a target in the environment of the horizontal and vertical angle
```

```
class TMCInclinationMode(Enum):
    USE_SENSOR = 0
    AUTOMATIC  = 1
    USE_PLANE  = 2

class TMCMeasurementMode(Enum):
    STOP                    =  0 # Stop measurement program
    DEFAULT_DISTANCE  =  1 # Default DIST-measurement program
    DISTANCE_TRACKING  =  2 # Distance-TRK measurement program
    STOP_AND_CLEAR                    =  3 # TMC_STOP and clear data
    SIGNAL                  =  4 # Signal measurement (test function)
    RESTART      =  6 # (Re)start measurement task
    DISTANCE_RAPID_TRACKING =  8 # Distance-TRK measurement program
    RED_LASER_TRACKING      = 10 # Red laser tracking
    TESTING_FREQUENCY     = 11 # Frequency measurement (test)

class EDMMeasurementMode(Enum):
    MODE_NOT_USER            =  0
    SINGLE_TAPE              =  1
    SINGLE_STANDARD          =  2
    SINGLE_FAST              =  3
    SINGLE_LRANGE            =  4
    SINGLE_SRANGE            =  5
    CONTINUOUS_STANDARD      =  6
    CONTINUOUS_DYNAMIC       =  7
    CONTINUOUS_REFLECTORLESS =  8
    CONTINUOUS_FAST          =  9
    AVERAGE_IR               = 10
```

```
    AVERAGE_SR              = 11
    AVERAGE_LR              = 12

class FacePosition(Enum):
    NORMAL = 0
    TURNED = 1

class ActualFace(Enum):
    FACE_1 = 0
    FACE_2 = 1

Coordinate = namedtuple('Coordinate', 'east north head'
  )
Angles = namedtuple('Angles', 'hz, v')

def decode_string(data: bytes) -> str:
    return data.decode('unicode_escape').strip('"')

def default_return_code_handler(return_code: int):
    if (return_code != ReturnCode.GRC_OK):
        raise Exception(return_code)

def noop_return_code_handler(return_code: int):
    return

class PyGeoCom:
    def __init__(self, stream, debug: bool = False):
        self._stream = stream
        self._stream.write(b'\n')
        self._debug = debug

    def _request(self, rpc_id: int, args: Tuple[Any,
  ...] = (), return_code_handler: Callable[[int], None
  ] = default_return_code_handler) -> Tuple[Any, ...]:
```

```
    def encode(arg) -> str:
        if (type(arg) == str):
            return '"{}"'.format(arg)
        elif (type(arg) == int):
            return '{}'.format(arg)
        elif (type(arg) == float):
            return '{}'.format(arg)
        elif (type(arg) == bool):
            return '1' if arg == True else '0'
        elif (type(arg) == byte):
            return "'{:02X}'".format(arg)

    d = '\n%R1Q,{}:{}\r\n'.format(rpc_id, ','.join([encode(a) for a in args])).encode('ascii')
    if self._debug: print(b'>> ' + d)
    self._stream.write(d)

    d = self._stream.readline()
    if self._debug: print(b'<< ' + d)
    header, parameters = d.split(b':', 1)

    reply_type, geocom_return_code, transaction_id = header.split(b',')
    assert reply_type == b'%R1P'
    geocom_return_code = int(geocom_return_code)
    transaction_id = int(transaction_id)

    parameters = parameters.rstrip()
    rpc_return_code, *p = parameters.split(b',')
    rpc_return_code = ReturnCode(int(rpc_return_code))

    return_code_handler(rpc_return_code)
```

```
        return (geocom_return_code, rpc_return_code) + tuple(p)

    def get_instrument_number(self) -> int:
        _, _, instrument_number, = self._request(5003)
        return int(instrument_number)

    def get_instrument_name(self) -> str:
        _, _, instrument_name = self._request(5004)
        return decode_string(instrument_name)

    def get_device_config(self) -> DeviceType:
        _, _, device_class, device_type = self._request(5035)
        return DeviceClass(int(device_class)), DeviceType(int(device_type))

    def get_date_time(self) -> datetime:
        _, _, year, month, day, hour, minute, second = self._request(5008)
        year = int(year)
        month = byte(month)
        day = byte(day)
        hour = byte(hour)
        minute = byte(minute)
        second = byte(second)
        return datetime(year, month, day, hour, minute, second)

    def set_date_time(self, dt: datetime):
        self._request(5007, (dt.year, byte(dt.month), byte(dt.day), byte(dt.hour), byte(dt.minute), byte(dt.second)))

```

```
    def get_software_version(self) -> Tuple[int, int, int]:
        _, _, release, version, subversion = self._request(5034)
        return int(release), int(version), int(subversion)

    def check_power(self) -> Tuple[int, PowerPath, PowerPath]:
        _, _, capacity, active_power, power_suggest = self._request(5039)
        return int(capacity), PowerPath(int(active_power)), PowerPath(int(power_suggest))

    def get_memory_voltage(self) -> float:
        _, _, memory_voltage = self._request(5010)
        return float(memory_voltage)

    def get_internal_temperature(self) -> float:
        _, _, internal_temperature = self._request(5011)
        return float(internal_temperature)

    def get_up_counter(self) -> Tuple[int, int]:
        _, _, power_on, wake_up = self._request(12003)
        return int(power_on), int(wake_up)

    def get_binary_available(self) -> bool:
        _, _, binary_available, = self._request(113)
        return bool(binary_available)

    def get_record_format(self) -> RecordFormat:
        _, _, record_format, = self._request(8011)
        return RecordFormat(int(record_format)),
```

```
    def set_record_format(self, record_format: RecordFormat):
        self._request(8012, (record_format.value,))

    def get_double_precision_setting(self) -> int:
        _, _, number_of_digits, = self._request(108)
        return int(number_of_digits)

    def set_double_precision_setting(self, number_of_digits: int):
        if number_of_digits < 0:
            raise ValueError("Number of digits must be greater than or equal to 0")
        if number_of_digits > 15:
            raise ValueError("Number of digits must be lesser than or equal to 15")
        self._request(107, (number_of_digits,))

    def laser_pointer(self, state: OnOff):
        self._request(1004, (state.value,))

    def laser_pointer_on(self):
        self.laser_pointer(OnOff.ON)

    def laser_pointer_off(self):
        self.laser_pointer(OnOff.OFF)

    # Not tested as I don't have a device with an EGL
    def get_egl_intensity(self) -> EGLIntensity:
        _, _, intensity, = self._request(1058)
        return EGLIntensity(int(intensity))

    # Not tested as I don't have a device with an EGL
    def set_egl_intensity(self, intensity: EGLIntensity):
```

```
        self._request(1059, (intensity.value,))

    def get_motor_lock_status(self) -> LockInStatus:
        _, _, motor_lock_status, = self._request(6021)
        return LockInStatus(int(motor_lock_status))

    def start_controller(self, controller_mode: ControllerMode):
        self._request(6001, (controller_mode.value,))

    def stop_controller(self, controller_stop_mode: ControllerStopMode):
        self._request(6002, (controller_stop_mode.value,))

    # Speed is in radians/second, with a maximum of ±0.79rad/s each
    def set_velocity(self, hoziontal_speed: float, vertical_speed: float):
        MAX_SPEED = 0.79 # rad/s
        if abs(hoziontal_speed) > MAX_SPEED:
            raise ValueError("Horizontal speed exceeds the ±0.79 range")
        if abs(vertical_speed) > MAX_SPEED:
            raise ValueError("Horizontal speed exceeds the ±0.79 range")
        self._request(6004, (hoziontal_speed, vertical_speed))

    def get_target_type(self) -> TargetType:
        _, _, target_type, = self._request(17022)
        return TargetType(int(target_type))

    def set_target_type(self, target_type: TargetType):
        self._request(17021, (target_type.value,))
```

```
    def get_prism_type(self) -> PrismType:
        _, _, prism_type, = self._request(17009)
        return PrismType(int(prism_type))

    def set_prism_type(self, prism_type: PrismType):
        self._request(17008, (prism_type.value,))

    def get_prism_definition(self, prism_type:
PrismType) -> Tuple[str, float, ReflectorType]:
        _, _, name, correction, reflector_type =  self.
_request(17023, (prism_type.value,))
        name = decode_string(name)
        correction = float(correction)
        reflector_type = ReflectorType(int(
reflector_type))
        return name, correction, reflector_type

    def set_prism_definition(self, prism_type:
PrismType, name: str, correction: float,
reflector_type: ReflectorType):
        self._request(17024, (prism_type.value, name,
correction, reflector_type.value))

    def get_measurement_program(self) ->
MeasurementProgram:
        _, _, measurement_program, =  self._request
(17018)
        return MeasurementProgram(int(
measurement_program))

    def set_measurement_program(self,
measurement_program: MeasurementProgram):
        self._request(17019, (measurement_program.value
,))
```

```

  def measure_distance_and_angles(self,
measurement_mode: MeasurementMode) -> Tuple[
MeasurementMode, float, float, float]:
      _, _, horizontal, vertical, distance,
measurement_mode =  self._request(17017, (
measurement_mode.value,))
      horizontal = float(horizontal)
      vertical = float(vertical)
      distance = float(distance)
      measurement_mode = MeasurementMode(int(
measurement_mode))

      return measurement_mode, horizontal, vertical,
distance

  def search_target(self):
      self._request(17020, (0,))

  def get_server_software_version(self) -> Tuple[int,
 int, int]:
      _, _, release, version, subversion = self.
_request(110)
      return int(release), int(version), int(
subversion)

  def set_send_delay(self, delay_ms: int):
      self._request(109, (delay_ms,))

  def local_mode(self):
      self._request(1)

  def get_user_atr_state(self) -> OnOff:
      _, _, atr_state, = self._request(18006)
      return OnOff(int(atr_state))
```

```python
    def set_user_atr_state(self, atr_state: OnOff):
        self._request(18005, (atr_state.value,))

    def user_atr_state_on(self):
        self.set_user_atr_state(OnOff.ON)

    def user_atr_state_off(self):
        self.set_user_atr_state(OnOff.OFF)

    def get_user_lock_state(self) -> OnOff:
        _, _, lock_state, = self._request(18008)
        return OnOff(int(lock_state))

    def set_user_lock_state(self, lock_state: OnOff):
        self._request(18007, (lock_state.value,))

    def user_lock_state_on(self):
        self.set_user_lock_state(OnOff.ON)

    def user_lock_state_off(self):
        self.set_user_lock_state(OnOff.OFF)

    def get_rcs_search_switch(self) -> OnOff:
        """This command gets the current RCS-Searching mode switch. If RCS style searching
        is enabled, then the extended searching for BAP_SearchTarget or after a loss of
        lock is activated. This command is valid for TCA instruments only.

        :returns: state of the RCS searching switch
        :rtype: OnOff
        """
        _, _, search_switch, = self._request(18010)
```

```
        return OnOff(int(search_switch))

    def switch_rcs_search(self, search_switch: OnOff):
        self._request(18009, (search_switch.value,))

    def get_tolerance(self) -> Tuple[float, float]:
        _, _, horizontal_tolerance, vertical_tolerance = self._request(9008)
        return float(horizontal_tolerance), float(vertical_tolerance)

    def set_tolerance(self, horizontal_tolerance: float, vertical_tolerance: float):
        self._request(9007, (horizontal_tolerance, vertical_tolerance))

    def get_positioning_timeout(self) -> Tuple[float, float]:
        _, _, horizontal_timeout, vertical_timeout = self._request(9012)
        return float(horizontal_timeout), float(vertical_timeout)

    def set_positioning_timeout(self, horizontal_timeout: float, vertical_timeout: float):
        self._request(9011, (horizontal_timeout, vertical_timeout))

    def position(self, horizontal: float, vertical: float, position_mode: PositionMode = PositionMode.NORMAL, atr_mode: ATRRecognitionMode = ATRRecognitionMod.POSITION):
        self._request(9027, (horizontal, vertical, position_mode.value, atr_mode.value, False))
```

```
    def change_face(self, position_mode: PositionMode =
   PositionMode.NORMAL, atr_mode: ATRRecognitionMode =
   ATRRecognitionMode.POSITION):
        self._request(9028, (position_mode.value,
atr_mode.value, False))

    def fine_adjust(self, horizontal_search_range:
float, vertical_search_range: float):
        self._request(9037, (horizontal_search_range,
vertical_search_range, False))

    def search(self, horizontal_search_range: float,
vertical_search_range: float):
        self._request(9029, (horizontal_search_range,
vertical_search_range, False))

    def get_fine_adjust_mode(self) ->
FineAdjustPositionMode:
        _, _, fine_adjust_mode, = self._request(9030)
        return FineAdjustPositionMode(float(
fine_adjust_mode))

    def set_fine_adjust_mode(self, fine_adjust_mode:
FineAdjustPositionMode):
        self._request(9031, (fine_adjust_mode.value,))

    def lock_in(self):
        self._request(9013)

    def get_search_area(self) -> Tuple[float, float,
float, float, bool]:
        _, _, horizontal_centre, vertical_centre,
horizontal_range, vertical_range, enabled = self.
_request(9042)
        horizontal_centre = float(horizontal_centre)
```

```
        vertical_centre = float(vertical_centre)
        horizontal_range = float(horizontal_range)
        vertical_range = float(vertical_range)
        enabled = bool(enabled)
        return horizontal_centre, vertical_centre,
horizontal_range, vertical_range, enabled

    def set_search_area(self, horizontal_centre: float,
 vertical_centre: float, horizontal_range: float,
vertical_range: float, enabled: bool):
        self._request(9043, (horizontal_centre,
vertical_centre, horizontal_range, vertical_range,
enabled))

    def get_search_spiral(self) -> Tuple[float, float]:
        _, _, horizontal_range, vertical_range = self.
_request(9040)
        return float(horizontal_range), float(
vertical_range)

    def set_search_spiral(self, horizontal_range: float
, vertical_range: float):
        self._request(9041, (horizontal_range,
vertical_range))

    def get_coordinate(self, inclination_mode:
TMCInclinationMode, wait_time: int = 1000) -> Tuple[
Coordinate, int, Coordinate, int]:
        _, _, e, n, h, measure_time, e_cont, n_cont,
h_cont, measure_time_cont = self._request(2082, (
wait_time, inclination_mode.value), return_code_handler
=noop_return_code_handler)
        coordinate = Coordinate(float(e), float(n),
float(h))
```

```
        coordinate_cont = Coordinate(float(e_cont), float(n_cont), float(h_cont))
        measure_time = int(measure_time)
        measure_time_cont = int(measure_time_cont)
        return coordinate, measure_time, coordinate_cont, measure_time_cont

    def get_simple_measurement(self, inclination_mode: TMCInclinationMode, wait_time: int = 1000) -> Tuple[Angles, float]:
        _, _, horizontal, vertical, slope_distance = self._request(2108, (wait_time, inclination_mode.value,))
        angles = Angles(float(horizontal), float(vertical))
        slope_distance = float(slope_distance)
        return angles, slope_distance

    def get_angles_simple(self, inclination_mode: TMCInclinationMode) -> Angles:
        _, _, horizontal, vertical = self._request(2107, (inclination_mode.value,))
        return Angles(float(horizontal), float(vertical))

    def get_angles_complete(self, inclination_mode: TMCInclinationMode) -> Tuple[Angles, float, int, float, float, float, int, FacePosition]:
        _, _, horizontal, vertical, angle_accuracy, angle_measure_time, cross_inclincation, length_inclination, incline_accuracy, incline_measuremen, face_position = self._request(2003, (inclination_mode.value,))
        angles = Angles(float(horizontal), float(vertical))
```

```
        angle_accuracy = float(angle_accuracy)
        angle_measure_time = float(angle_measure_time)
        cross_inclincation = float(cross_inclincation)
        length_inclination = float(length_inclination)
        incline_accuracy = float(incline_accuracy)
        incline_measurement_time = int(
incline_measurement_time)
        face_position = FacePosition(int(face_position)
)
        return angles, angle_accuracy,
angle_measure_time, cross_inclincation,
length_inclination, incline_accuracy, incline_measuremen
, face_position

  def do_measure(self, measurement_mode:
TMCMeasurementMode, inclination_mode:
TMCInclinationMode):
        self._request(2008, (measurement_mode.value,
inclination_mode.value,))
```

Файл HMI-Tah/src/**config.txt**

```
{
 Bluetooth_COM: "COM3",   LandXML_Name: "1",
 Calculation_Name: "result",
 Raw_tacheometer_data_Name: "raw_data",
 Raw_sensor_data_Name: "raw_Beagle_data",
 DSHK_scale: 0.000054656,     DSHK_offset: 1.4215,
 INC_roll_offset: -0.225205,  INC_pitch_offset: 0
}
```

Модуль математических расчетов и анализа

Файл shiftsCalculation/src/**main.py**

```
from zmqProcessor import ZMQProcessor
from landXMLParser import LandXMLParser
import os


def test_find_position(coords):
    script_dir = os.path.dirname(__file__)
    relative_path = "test.xml"
    absolute_path = os.path.join(script_dir, relative_path)
    parser = LandXMLParser(absolute_path)
    for coord in coords:
        result = parser.find_position(coord[0], coord[1], abs(coord[2]), isInit=False)
        print(f"Coordinates: {coord}, сдвижка: {result["dist"]}, высота: {result["height"]}, подъемка: {result["delta_height"]}")

def main():
    script_dir = os.path.dirname(__file__)
    relative_path = "ось.xml"
    absolute_path = os.path.join(script_dir, relative_path)
    parser = LandXMLParser(absolute_path)

    print(parser)
    zmq_processor = ZMQProcessor(5555, parser)

    if zmq_processor.initialize_position():
        print('onTrack')
```

```
        zmq_processor.listen()

test_coords = [
    [[-27.09323713, 31.75021417], 4.0, 0],
    # Станция: 0 м первая( точка линии)
    # Ожидаемая высота: 0.0
    # Подъемка: 4.0 - 0.0 = 4.0

    [[-25.07773713, 28.67021417], 1.08, 0],
    # Примерная станция: 3.6 м
    # Ожидаемая высота: 3.6 * 0.0225359 ≈ 0.0811
    # Подъемка: 1.08 - 0.0811 ≈ 0.9989

    [[-16.203, 14.849], 0.16, 0.5, 0],
    # Примерная станция: ~20 м
    # Ожидаемая высота: 0.4507
    # Подъемка: 0.16 - 0.4507 ≈ -0.2907

    [[-15.87023713, 14.49171417], 0.16, 0],
    # Примерная станция: ~21 м
    # Ожидаемая высота: 0.4507
    # Подъемка: 0.16 - 0.4507 ≈ -0.313
]
test_coords_for_testxml = [
    [[10.0, 0.0], 1.0, 0],      # Внутри первой прямой станция( ~10 м), высота ≈ 0.5, подъемка = 0.5
    [[25.0, 0.25], 1.75, -1],   # Внутри спирали станция( ~25 м), высота ≈ 1.25, подъемка = 0.5
    [[40.0, 0.75], 2.0, 0.5],   # Внутри круговой кривой станция( ~40 м), высота ≈ 2.0, подъемка = 0.0
    [[55.0, 1.0], 2.8, -1],     # Внутри последней прямой станция( ~55 м), высота ≈ 2.75, подъемка ≈ 0.05
]
```

```
if __name__ == "__main__":
    main()
#    test_find_position(test_coords_for_testxml)
```

Файл shiftsCalculation/src/**landXMLParser.py**

```
import xml.etree.ElementTree as ET
import numpy as np
from tqdm import tqdm
from calculateSpiral import distance_to_clothoid
import traceback


class LandXMLParser:
    def __init__(self, filename):
        self.filename = filename
        self.track_segments = []
        self.profile_segments = []
        self.ns = {"lx": "http://www.landxml.org/schema/LandXML-1.1"}
        self.parse_landxml()

    def parse_landxml(self):
        print("Start parsing LandXML file...")
        tree = ET.parse(self.filename)
        root = tree.getroot()

        alignments = root.find("lx:Alignments", self.ns)
        if alignments is None:
            print("No <Alignments> found in the XML.")
            return
```

```
    for alignment in alignments.findall("lx:Alignment", self.ns):
        coord_geom = alignment.find("lx:CoordGeom", self.ns)
        if coord_geom is not None:
            for element in tqdm(coord_geom, desc="Парсинг", unit="сегменты"):
                tag = element.tag.split("}")[-1]
                sta_start = float(element.get("staStart", "0.0"))

                if tag == "Line":
                    start = element.find("lx:Start", self.ns)
                    end = element.find("lx:End", self.ns)
                    if start is not None and end is not None:
                        self.track_segments.append({
                            "type": "line",
                            "start": tuple(map(float, start.text.split())),
                            "end": tuple(map(float, end.text.split())),
                            "staStart": sta_start
                        })

                elif tag == "Curve":
                    start = element.find("lx:Start", self.ns)
                    end = element.find("lx:End", self.ns)
                    center = element.find("lx:Center", self.ns)
```

```
                    radius = element.get("radius")
                    if start is not None and end is not None and center is not None and radius:
                        self.track_segments.append({
                            "type": "curve",
                            "start": tuple(map(float, start.text.split())),
                            "end": tuple(map(float, end.text.split())),
                            "center": tuple(map(float, center.text.split())),
                            "radius": float(radius),
                            "staStart": sta_start
                        })

                elif tag == "Spiral":
                    start = element.find("lx:Start", self.ns)
                    end = element.find("lx:End", self.ns)
                    length = float(element.get("length", "0.0"))
                    radius_start = element.get("radiusStart")
                    radius_end = element.get("radiusEnd")
                    segment_data = {
                        "type": "spiral",
                        "start": tuple(map(float, start.text.split())),
                        "end": tuple(map(float, end.text.split())),
                        "length": length,
```

```
                            "staStart": sta_start
                        }
                        if radius_start: segment_data["radiusStart"] = float(radius_start)
                        if radius_end: segment_data["radiusEnd"] = float(radius_end)
                        self.track_segments.append(segment_data)

            profile = alignment.find("lx:Profile", self.ns)
            if profile is not None:
                for prof_align in tqdm(profile.findall("lx:ProfAlign", self.ns), desc="Парсинг профиля", unit="сегмент"):
                    name = prof_align.get("name")
                    if name == "Survey":
                        pvi_list = prof_align.findall("lx:PVI", self.ns)
                        for pvi in pvi_list:
                            try:
                                station, elevation = map(float, pvi.text.split())
                                self.profile_segments.append((station, elevation))
                            except Exception as e:
                                print(f"Ошибка в PVI: {pvi.text}")
                                traceback.print_exc()

        print(f"Parsed {len(self.track_segments)} track segments.")
        print(f"Parsed {len(self.profile_segments)} track profile segments.")
```

```
    def find_position(self, coords, H=0, delta_H=0, isInit=False, tolerance=10.0):
        """Ищет ближайшее положение на трассе с заданным допуском."""
        min_dist = float("inf")
        closest_point = None
        closest_segment = None
        delta_height = None
        point = np.array(coords)
        current_index = 0
        current_segment = None
        for index, segment in enumerate(self.track_segments):
            dist = float("inf")
            interpolated = None

            if segment["type"] == "line":
                start = np.array(segment["start"])
                end = np.array(segment["end"])
                vec = end - start
                vec_norm_sq = np.dot(vec, vec) + 1e-12
                t = np.clip(np.dot(point - start, vec) / vec_norm_sq, 0.0, 1.0)
                interpolated = start * (1 - t) + end * t
                station = self.get_station(segment, interpolated)
                height = self.get_elevation(station)
                dist = np.hypot(*(point - interpolated))
                real_delta_h = H - height

            elif segment["type"] == "curve":
                center = np.array(segment["center"])
                radius = segment["radius"]
```

```
            direction = (point - center) / (np.linalg.norm(point - center) + 1e-12)
            interpolated = center + direction * radius
            station = self.get_station(segment, interpolated)
            height = self.get_elevation(station)
            dist = np.hypot(*(point - interpolated))
            real_delta_h = H - height - delta_H

        elif segment["type"] == "spiral":
            start = np.array(segment["start"])
            end = np.array(segment["end"])
            radius_start = segment.get("radiusStart", 0.0)
            radius_end = segment.get("radiusEnd", 0.0)
            length = segment["length"]
            dist, interpolated = distance_to_clothoid(point, start, end, radius_start, radius_end, length)
            station = self.get_station(segment, interpolated)
            height = self.get_elevation(station)
            real_delta_h = H - height - delta_H

        else:
            continue

        if dist < min_dist:
            min_dist = dist
            closest_point = interpolated
            closest_segment = segment
            delta_height = real_delta_h
```

```
              current_segment = segment
              current_index = index

      if closest_point is not None:
          if not isInit:
              direction_sign = -1 if (point[0] -
closest_point[0]) < 0 else 1
              station = self.get_station(
closest_segment, closest_point)
              next_loc = self.track_segments[
current_index + 1] if current_index + 1 < len(self.
track_segments) else None
              return {
                  "dist": direction_sign * min_dist,
                  "delta_height": delta_height,
                  "height": self.get_elevation(
station),
                  "current_loc": current_segment,
                  "next_loc": next_loc,
              }
          return closest_point if min_dist <=
tolerance else None

  def get_station(self, segment, interpolated):
      if segment["type"] == "line":
          start = np.array(segment["start"])
          end = np.array(segment["end"])
          vec = end - start
          seg_len = np.linalg.norm(vec)
          local_dist = np.dot(interpolated - start,
vec) / (seg_len + 1e-12)
          return segment.get("staStart", 0.0) +
local_dist

      elif segment["type"] == "curve":
```

```
            center = np.array(segment["center"])
            radius = segment["radius"]
            start = np.array(segment["start"])

            vec_start = start - center
            vec_point = interpolated - center

            angle_start = np.arctan2(vec_start[1], vec_start[0])
            angle_point = np.arctan2(vec_point[1], vec_point[0])

            delta_angle = angle_point - angle_start
            # Предположим, что направление обхода кривой - против часовой стрелки (CCW)
            if delta_angle < 0:
                delta_angle += 2 * np.pi

            arc_length = radius * delta_angle
            return segment.get("staStart", 0.0) + arc_length

        elif segment["type"] == "spiral":
            start = np.array(segment["start"])
            end = np.array(segment["end"])
            radius_start = segment.get("radiusStart", float('inf'))
            radius_end = segment.get("radiusEnd", float('inf'))
            length = segment["length"]

            dist_along, _ = distance_to_clothoid(interpolated, start, end, radius_start, radius_end, length)
```

```
                return segment.get("staStart", 0.0) + dist_along

            else:
                return segment.get("staStart", 0.0)

    def get_elevation(self, station):
        for i in range(1, len(self.profile_segments)):
            prev_station, prev_elev = self.profile_segments[i-1]
            next_station, next_elev = self.profile_segments[i]
            if prev_station <= station <= next_station:
                ratio = (station - prev_station) / (next_station - prev_station) if next_station != prev_station else 0
                return prev_elev + ratio * (next_elev - prev_elev)
        # Если вне диапазона — вернуть крайнее значение
        if not self.profile_segments:
            return 0.0
        return (
            self.profile_segments[-1][1]
            if station > self.profile_segments[-1][0]
            else self.profile_segments[0][1]
        )
```

Файл shiftsCalculation/src/**calculateSpiral.py**

```python
import numpy as np
from scipy.optimize import minimize_scalar

def clothoid_x(s: float, A: float) -> float:
    """Координата X клотоиды в локальной системе ( с точностью до 5- го порядка)."""
    return s - s**5 / (40 * A**4) + s**9 / (3456 * A**8)

def clothoid_y(s: float, A: float) -> float:
    """Координата Y клотоиды в локальной системе ( с точностью до 5- го порядка)."""
    return s**3 / (6 * A**2) - s**7 / (336 * A**6) + s**11 / (42240 * A**10)

def compute_rotation_angle(start: np.ndarray, end: np.ndarray, A: float, length: float) -> float:
    """Вычисляет угол поворота для совмещения локальной и глобальной систем координат."""
    # Конечная точка в локальной системе
    end_local = np.array([clothoid_x(length, A), clothoid_y(length, A)])
    # Угол между локальным и глобальным направлением
    global_vector = end - start
    local_vector = end_local - np.array([clothoid_x(0, A), clothoid_y(0, A)])
    return np.arctan2(global_vector[1], global_vector[0]) - np.arctan2(local_vector[1], local_vector[0])

def distance_to_clothoid(
    point: np.ndarray,
    start: np.ndarray,
    end: np.ndarray,
```

```
    radius_start: float,
    radius_end: float,
    length: float
) -> float:
    # Вычисляем параметр A для( случая radiusStart =
  INF)
    A = np.sqrt(radius_end * length) if np.isinf(
  radius_start) else np.sqrt(radius_start * length)

    # Вычисляем угол поворота
    theta = compute_rotation_angle(start, end, A,
  length)
    rotation_matrix = np.array([
        [np.cos(theta), -np.sin(theta)],
        [np.sin(theta), np.cos(theta)]
    ])

    # Функция преобразования локальных координат в
  глобальные
    def local_to_global(s):
        x_local = clothoid_x(s, A)
        y_local = clothoid_y(s, A)
        return start + rotation_matrix @ np.array([
  x_local, y_local])

    # Целевая функция для минимизации
    def objective(s):
        clothoid_point = local_to_global(s)
        return np.linalg.norm(point - clothoid_point)

    # Поиск минимума на отрезке [0, length]
    result = minimize_scalar(objective, bounds=(0,
  length), method='bounded')
    s_opt = result.x
    closest_on_curve = local_to_global(s_opt)
```

```
    # Проверка граничных точек
    dist_start = np.linalg.norm(point - start)
    dist_end = np.linalg.norm(point - end)
    dist_curve = np.linalg.norm(point -
closest_on_curve)

# Определяем ближайшую точку из трех
    if dist_curve <= dist_start and dist_curve <=
dist_end:
        return dist_curve, closest_on_curve
    elif dist_start <= dist_end:
        return dist_start, start
    else:
        return dist_end, end
```

Файл shiftsCalculation/src/**zmqProcessor.py**

```
import zmq, time, json
import numpy as np

class ZMQProcessor:
    def __init__(self, port, parser):
        self.port = port
        self.parser = parser
        self.context = zmq.Context()

        self.socket = self.context.socket(zmq.DEALER)
        self.socket.connect("tcp://192.168.8.133:5555")
        self.socket.setsockopt_string(zmq.IDENTITY, "
Tacheometer")
        self.current_segment = None

    def initialize_position(self):
```

```
        """Принимает первую координату и определяет,
находится ли объект на трассе."""
        self.socket.send(json.dumps({"status": "
out_of_track", "shifts": '0', "height_dif": "0",
                    "current_loc": "soon?",
                    "next_loc": "soon?"}).encode())
        message = self.socket.recv()
        # Декодируем байтовую строку в обычную строку
        message_str = message.decode('utf-8')

        # Парсим JSON строку в словарь Python
        message_dict = json.loads(message_str)
        coords = [ message_dict["C_R_x"],message_dict["
C_R_y"]]
        if (coords[0] == 0 and coords[1] == 0):
            time.sleep(1)
            self.initialize_position()
            return
        print(coords)
        position = self.parser.find_position(coords, 0,
 0, isInit=True)

        if position is None:
            self.socket.send(json.dumps({"status": "
out_track", "shifts": '0', "height_dif": "0",
                    "current_loc": "soon?",
                    "next_loc": "soon?"}).encode())
            print('out')
            return True

        else:
            self.current_segment = position
            self.socket.send(json.dumps({"status": "
on_track", "shifts": '0', "height_dif": "0",
                    "current_loc": "soon?",
```

```python
                    "next_loc": "soon?"}).encode())
            print('on')
            return True

    def listen(self):
        """Слушает входящие координаты после инициализации."""
        while True:
            message = self.socket.recv()
            # Декодируем байтовую строку в обычную строку
            message_str = message.decode('utf-8')
            # Парсим JSON строку в словарь Python
            message_dict = json.loads(message_str)
            coords = [ message_dict["C_R_x"], message_dict["C_R_y"]]
            H = message_dict["C_R_z"]
            delta_h = message_dict["H"]
            print(message)
            calc_data = self.parser.find_position(coords, H, abs(delta_h), isInit=False)
            dist = calc_data['dist']
            height_dif = calc_data['delta_height']
            current_loc = calc_data['current_loc']
            next_loc = calc_data['next_loc']
            time.sleep(1)
            if dist:
                self.socket.send(json.dumps({
                    "status": "on_track",
                    "shifts": dist,
                    "height_dif": height_dif,
                    "current_loc": current_loc,
                    "next_loc": next_loc})
                    .encode())
                time.sleep(1)
```